UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interacciones complejas entre conjuntos de elementos y distintos niveles de incertidumbre. Dentro de este dominio, el framework MDP-ProbLog utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural ya que la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para describir variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, poco práctico y propenso a inconsistencias semánticas.

Se reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes . . . 2
- 1.2. Definición del Problema . . . 4
- 1.3. Objetivos . . . 5
  - 1.3.1. Objetivo General . . . 5
  - 1.3.2. Objetivos Específicos . . . 5
- 1.4. Justificación . . . 6
- 1.5. Alcances y Limitaciones . . . 7
  - 1.5.1. Alcances . . . 7
  - 1.5.2. Limitaciones . . . 8

2. Estado del Arte 9
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 9
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 10
- 2.3. Disyunciones Anotadas como mecanismo de exclusión mutua . . . 11
- 2.4. Compilación de conocimiento para inferencia eficiente . . . 11
- 2.5. Análisis Comparativo . . . 12
- 2.6. Conclusión del Estado del Arte . . . 13

3. Marco Teórico 14
- 3.1. Grid Estocástico . . . 14
- 3.2. Programación Lógica . . . 15
- 3.3. Programación Lógica Probabilística . . . 16
  - 3.3.1. Semántica de distribución . . . 16
- 3.4. Lenguaje ProbLog . . . 18
- 3.5. Disyunciones Anotadas . . . 19
- 3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento . . . 21
- 3.7. Procesos de Decisión de Markov . . . 22
- 3.8. MDPs Factorizados y Fluentes de Estado . . . 24
- 3.9. El framework MDP-ProbLog . . . 24

4. Diseño e Implementación 26
- 4.1. Análisis de la arquitectura original . . . 26
  - 4.1.1. Componentes del sistema original . . . 26
  - 4.1.2. Flujo de ejecución . . . 28
  - 4.1.3. Limitaciones en la gestión de fluentes . . . 28

4.2. Esquema factorizado del espacio de estados . . . 29
4.2.1. Modelo de datos . . . 29
4.2.2. Codificación *mixed-radix* . . . 30
4.2.3. Ejemplo mixed-radix . . . 31
4.2.4. Operaciones relevantes . . . 31
4.3. Clasificador automático de fluentes . . . 32
4.3.1. Modos de declaración . . . 32
4.3.2. El índice invertido de Disyunciones Anotadas . . . 32
4.3.3. Pipeline de clasificación . . . 33
4.3.4. Inferencia implícita de tipo . . . 34
4.4. Pipeline de preparación del MDP . . . 35
4.4.1. Fases del pipeline . . . 35
4.4.2. Hechos *dummy* y mecanismo de evidencia . . . 37
4.4.3. Corrección de la compilación doble . . . 37
4.4.4. Declaración de fluentes multivaluados . . . 38
4.5. Adaptación de la Iteración de Valor . . . 38
4.6. Evaluador de Darwiche . . . 40
4.6.1. El costo de la evaluación estándar . . . 41
4.6.2. La idea central: derivadas como respuestas . . . 41
4.6.3. El algoritmo de diferenciación en dos pasadas . . . 41
4.6.4. Implementación y activación . . . 43
4.7. Herramientas auxiliares . . . 44
4.7.1. Exportador de resultados . . . 44
4.7.2. Simulador de trayectorias . . . 45
4.8. Retrocompatibilidad . . . 46

5. Experimentación y Resultados **48**
5.1. Diseño experimental . . . 48
5.1.1. Dominios de evaluación . . . 48
5.1.2. Variables independientes . . . 49
5.1.3. Protocolo de medición . . . 49
5.2. Entorno de evaluación . . . 50
5.3. RQ1 — Corrección funcional . . . 50
5.3.1. Verificación en el dominio Mitchell . . . 50
5.4. RQ2 — Simplificación del modelado . . . 51
5.5. RQ3 — Impacto computacional . . . 53
5.5.1. Tiempos de ejecución . . . 53
5.5.2. Cociente de tiempos entre codificaciones . . . 54
5.5.3. Frontera de escalabilidad . . . 54
5.5.4. Comparación de *backends* . . . 55
5.5.5. Desglose de la fase de construcción . . . 55
5.6. Dominio Russell . . . 56
5.6.1. Verificación de la función de valor óptima . . . 56
5.6.2. Tiempos de ejecución . . . 57
5.7. Discusión . . . 58

5.7.1. RQ1 — Correctitud funcional 58
5.7.2. RQ2 — Simplificación del modelado 59
5.7.3. RQ3 — Impacto computacional 59
5.7.4. Análisis de cuellos de botella 60
5.7.5. Limitaciones de la evaluación 60

Índice de figuras **62**

Índice de cuadros **63**

Índice de algoritmos **64**

Referencias **65**

1. Introducción

Dentro del ámbito de la Inteligencia Artificial, la Programación Lógica Probabilística se ha consolidado como un paradigma expresivo y formalmente fundamentado para la representación del conocimiento bajo incertidumbre, al combinar la capacidad deductiva de la programación lógica con el tratamiento de la probabilidad [1]. Uno de los desarrollos más representativos de este paradigma es *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y que permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias.

Asimismo, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov (MDPs) se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [2], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. En su formulación tabular clásica, cada estado del sistema se trata como una entidad indivisible y las transiciones se almacenan en una tabla que lista explícitamente la probabilidad de pasar de cada estado a cada otro estado bajo cada acción posible. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño de dicha tabla crece de manera exponencial, un fenómeno que Bellman denominó la "maldición de la dimensionalidad" [2]. Esta particularidad, hace que tanto la especificación como la resolución de MDPs se vuelvan computacionalmente costosas conforme crece la complejidad del dominio [3].

Integrando la expresividad lógica de ProbLog con la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [4]. Esta herramienta codifica estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar o, de manera relacional y compacta MDPs de horizonte infinito, es decir, problemas de decisión secuencial en los que no hay un punto de terminación predefinido y donde el agente busca una estrategia óptima a largo plazo. En lugar de tratar cada estado como una entidad indivisible, MDP-ProbLog adopta una representación factorizada en la que se descompone el estado global en un conjunto de variables independientes denominadas *fluentes*, de modo que la dinámica del sistema se describe en términos de cómo cambia cada variable individual. Esta representación permite *especificar* el MDP de forma compacta y legible, aunque la resolución del problema mediante el algoritmo de iteración de valor continúa siendo enumerativa y, por tanto, sujeta a la explosión combinatoria del espacio de estados.

1.1. Antecedentes

La idea de utilizar lenguajes declarativos para especificar MDPs no surge con MDP-ProbLog. Boutilier, Dean y Hanks [5] propusieron a finales de la década de 1990 el uso de representaciones factorizadas para abordar la maldición de la dimensionalidad, describiendo las transiciones de estado mediante redes bayesianas dinámicas en lugar de tablas monolíticas. Posteriormente, herramientas como SPUDD [6] explotaron diagramas de decisión algebraicos para resolver MDPs factorizados de manera simbólica.

En el contexto de la Programación Lógica Probabilística, Nitti, Belle y De Raedt [7] demostraron que los lenguajes probabilísticos basados en lógica podían servir como especificación declarativa de problemas de planificación para dominios tanto discretos como continuos. MDP-ProbLog [4] se inscribe en esta línea al proponer una codificación completamente declarativa de MDPs de horizonte infinito dentro de la sintaxis de ProbLog, sustituyendo el muestreo por inferencia exacta mediante Conteo de Modelos Ponderados.

MDP-ProbLog ha sido adoptado como base tecnológica por el equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga en la Universidad Politécnica de Victoria, el cual lo ha empleado en el desarrollo de sistemas de navegación y toma de decisiones para vehículos autónomos [8]. Sin embargo, al tratarse de un framework que únicamente admite fluentes booleanos, la construcción de los modelos requiere adaptar la descripción del dominio a las limitaciones de la herramienta, en lugar de permitir su representación en términos naturales. Esto en sí mismo es una limitación estructural inherente del framework, ya que todos los fluentes de estado se definen exclusivamente como variables de Bernoulli independientes.

Para ilustrar el problema de manera concreta, considérese un dominio de navegación en cuadrícula, un escenario canónico en la literatura de MDPs [9, 10] en el cual, un agente debe desplazarse desde una posición inicial hasta una celda meta, seleccionando en cada paso una de cuatro acciones de movimiento. La posición del agente compone una partde del estado del sistema y puede describirse mediante dos variables: la fila y la columna que ocupa.

Debido a que cada fluente de estado solo admite dos valores (verdadero o falso), la posición de un agente no puede codificarse directamente como un par de coordenadas, ya que cada coordenada naturalmente toma más de dos valores. Por ello, la representación booleana de MDP-ProbLog resulta insuficiente.

En el caso del grid de 2×3, donde existen 6 celdas posibles, se requieren al menos 3 fluentes booleanos para distinguir 6 estados distintos utilizando únicamente variables de dos valores, ya que 2 variables solo permiten $2^2 = 4$ combinaciones, insuficientes para representar las 6 celdas. Con 3 variables booleanas se obtienen $2^3 = 8$ combinaciones, cantidad suficiente para asignar una combinación única a cada celda.

Como se observa en la Tabla 1, las 8 combinaciones que generan los 3 fluentes booleanos exceden las 6 celdas reales del grid. A lo largo del documento, se denominará *estados espurios* a las combinaciones sobrantes, es decir, configuraciones que el sistema de inferencia considera válidas pero que carecen de correspondencia física en el dominio.

Tabla 1: Codificación binaria de las 6 celdas de un grid de 2×3.

b_1	b_2	b_3	**Celda asignada**
0	0	0	Celda (1,1)
0	0	1	Celda (1,2)
0	1	0	Celda (1,3)
0	1	1	(sin correspondencia)
1	0	0	Celda (2,1)
1	0	1	Celda (2,2)
1	1	0	Celda (2,3)
1	1	1	(sin correspondencia)

La Figura 1 contrasta ambas representaciones del mismo grid: a la izquierda, la descripción mediante las variables de fila (x) y columna (y), que genera exactamente 6 estados; a la derecha, la codificación booleana mediante tres fluentes binarios (b_1, b_2, b_3), que produce 8 combinaciones de las cuales 2 son espurias.

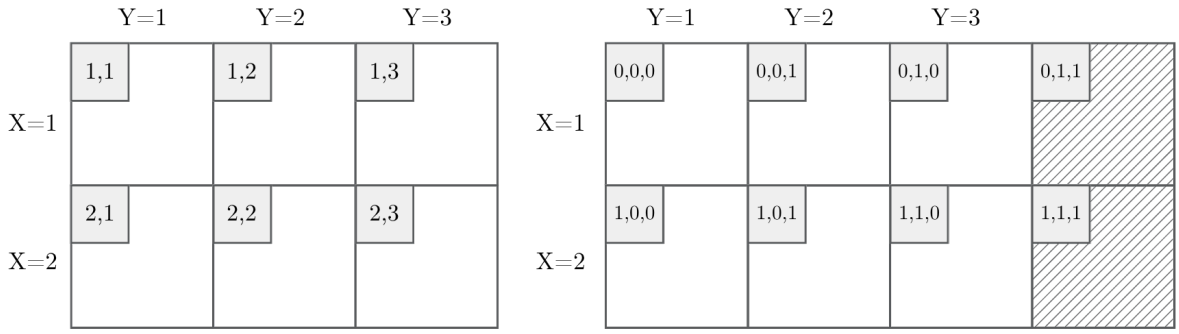


Figura 1: Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3.

Este procedimiento de codificación sigue un patrón general. Si se desea representar k estados mutuamente excluyentes se requieren al menos $\lceil \log_2(k) \rceil$ fluentes booleanos, los cuales generan $2^{\lceil \log_2(k) \rceil}$ combinaciones. Siempre que k no sea una potencia exacta de 2, el espacio de estados contendrá combinaciones sin significado claro y que el modelador debe gestionar manualmente. La Tabla 2 ilustra cómo este desajuste crece con el tamaño del dominio.

Tabla 2: Escalamiento de estados espurios en la codificación binaria por tamaño de grid.

Grid	**Celdas** (k)	**Bits necesarios**	**Combinaciones** (2^{bits})	**Espurios**
2×3	6	3	8	2
3×3	9	4	16	7
4×4	16	4	16	0
5×5	25	5	32	7
6×6	36	6	64	28

En un grid de 6×6, por ejemplo, 28 de las 64 combinaciones binarias son espurias, ejemplificando de manera clara las problematicas inherentes a los espacios de estados con variables categóricas mediante la representación que el framework impone actualmente.

1.2. Definición del Problema

Como se ilustró en la sección anterior con el grid de navegación, MDP-ProbLog carece de un mecanismo nativo para representar variables de estado que posean más de dos valores mutuamente excluyentes. Cada fluente se trata como una variable de Bernoulli independiente, lo que obliga a representar cualquier variable multivaluada mediante un conjunto de predicados booleanos cuyas combinaciones codifiquen los valores posibles.

Las consecuencias de esta restricción se extienden a cualquier dominio que requiera variables categóricas. Ejemplos típicos de esto, pueden ser, el estado de un semáforo, el carril que ocupa un vehículo o la fase de un proceso; todas ellas variables que, por su naturaleza, se describen mediante más de dos estados posibles.

En todos estos casos, para representar correctamente este tipo de variables, el usuario debe tener en cuenta estas limitaciones, mientras que el solver puede llegar a ejecutar la iteración de valor sobre un espacio de estados inflado con configuraciones inalcanzables, invirtiendo cómputo en estados que no contribuyen a la política resultante.

Para dimensionar cómo esta limitación afecta el modelado, es útil observar cómo herramientas históricas lidiaron con la misma barrera. La Tabla 3 muestra un ejemplo clásico en la literatura de MDPs factorizados: el dominio de planificación de procesos de manufactura (*Factory*), introducido para evaluar el algoritmo SPUDD [6]. En este escenario se requieren variables categóricas ternarias para modelar con precisión el entorno, tal como la calidad de la pintura de una pieza (buena, deficiente o inexistente).

Tabla 3: Codificación binaria forzada para una variable ternaria en el dominio *Factory*.

Bit 1	**Bit 2**	**Calidad de la pintura**
0	0	Buena (*good*)
0	1	Deficiente (*poor*)
1	0	Inexistente (*false*)
1	1	Estado espurio (*unreachable state*)

Al igual que ocurría en SPUDD, la codificación binaria en MDP-ProbLog introduce inevitablemente estados inalcanzables, lo que provoca que por un lado, wzista un aumento en la complejidad del modelado, debido a que el usuario no puede simplemente declarar los posibles valores de una variable. Esta limitación se traduce en una ineficiencia computacional.

En síntesis, la representación exclusivamente booleana produce modelos que son simultáneamente más costosos de escribir para el humano y menos eficientes de resolver para la máquina, forzando al dominio a adaptarse a las limitaciones de la herramienta.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando la exclusión mutua nativa de los valores de una variable.
- Desarrollar las herramientas de inspección que permitan examinar las matrices de transición, las tablas de recompensas y el historial de convergencia de los modelos resueltos.

1.4. Justificación

La extensión propuesta representa un paso necesario para la evolución del framework debido a las limitaciones impuestas por el modelado estrictamente binario. En ausencia de variables categóricas nativas, la arquitectura original transfiere al modelador la tarea de restringir matemáticamente el entorno. Es por ello que usuario debe diseñar y escribir reglas de integridad manuales para evitar que el sistema asigne valores contradictorios a una misma entidad (por ejemplo, evaluar a un agente que ocupa dos posiciones simultáneamente). Esta práctica introduce ruido en el código, oscureciendo la lógica real del proceso de decisión bajo capas de restricciones artificiales.

Se soluciona el problema al trasladar la carga de validación del humano hacia el motor lógico mediante las Disyunciones Anotadas, las cuales son un constructo nativo de la Programación Lógica Probabilística [11] que garantiza la exclusión mutua entre los valores de una variable por construcción semántica. Al delegar esta responsabilidad a la estructura interna de estas disyunciones, el código ProbLog resultante recobra legibilidad y el modelador concentra su esfuerzo en definir la dinámica y las recompensas del entorno, reduciendo la carga cognitiva, acortando los tiempos de desarrollo y eliminando una fuente recurrente de errores de diseño.

Además de las mejoras en la usabilidad, la precisión de esta nueva representación resuelve parte de las deficiencias en el rendimiento computacional. Durante el proceso de resolución original, el algoritmo de iteración de valor desperdicia ciclos de cómputo evaluando configuraciones inalcanzables, invirtiendo recursos en estados que jamás contribuirán a la política resultante. Al garantizar que el espacio de estados contenga únicamente los escenarios que el dominio contempla, el sistema elimina las configuraciones redundantes introducidas por la codificación booleana. Este ahorro computacional es directamente proporcional al número de estados espurios evitados; beneficio que se acumula considerablemente en dominios que utilizan múltiples variables categóricas.

Finalmente, la integración de las Disyunciones Anotadas para modelar fluentes multivaluados no requiere de la modificación de los algoritmos de compilación ni de razonamiento probabilístico. La propuesta extiende la expresividad de la herramienta sin comprometer su arquitectura subyacente ni su retrocompatibilidad con los modelos booleanos existentes.

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca el diseño, la implementación y la validación de una extensión al núcleo del framework MDP-ProbLog que incorpora soporte para fluentes de estado multivaluados. El desarrollo comprende los siguientes elementos:

1. **Representación factorizada del espacio de estados.** El rediseño de las estructuras de datos necesarias para que el espacio de estados admita factores con más de dos valores posibles, preservando la capacidad de iteración y la indexación eficiente sobre el conjunto completo de estados.

2. **Clasificación automática de fluentes.** Se implementó un mecanismo que determina automáticamente si un fluente declarado por el usuario es booleano o multivaluado, a partir del análisis de las Disyunciones Anotadas presentes en el programa. El usuario también puede especificar el tipo de forma explícita.

3. **Integración con el motor de inferencia.** Se adaptó la interfaz entre MDP-ProbLog y el motor de inferencia de ProbLog para soportar la inyección y el manejo de Disyunciones Anotadas como representación nativa de los fluentes multivaluados.

4. **Adaptación del algoritmo de resolución.** Se reestructuró el flujo de evaluación de transiciones y el algoritmo de Iteración de Valor para operar sobre transiciones agrupadas por factor, de modo que cada variable de estado se evalúa de acuerdo con su cardinalidad real.

5. **Herramientas de inspección.** Se desarrollaron utilidades para exportar las matrices de transición, las tablas de recompensas y el historial de convergencia, facilitando la verificación y el análisis de los modelos resueltos.

6. **Retrocompatibilidad.** La extensión preserva el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario.

La validación funcional se realiza sobre dominios basados en grids, específicamente el grid de Mitchell que constituye un problema canónico en la literatura de MDPs con soluciones óptimas conocidas. En cada dominio se contrastan dos formulaciones del mismo problema, una con codificación booleana y otra con fluentes multivaluados, evaluando la simplificación del proceso de modelado, la corrección de la política resultante y el comportamiento del solver en términos de convergencia y tamaño del espacio de estados.

1.5.2. Limitaciones

El desarrollo se restringe a MDPs de horizonte infinito con espacios de estados discretos y completamente observables. Los siguiente puntos delimitan el alcance técnico de la extensión:

1. **Enumeración explícita del espacio de estados.** El alcance de esta extensión se limita a la capa de representación del espacio de estados; la estrategia de resolución enumerativa del framework original permanece sin modificación. El algoritmo de Iteración de Valor continúa recorriendo todos los pares estado-acción en cada iteración, con un costo proporcional al número total de estados por el número de acciones. Dominios con múltiples variables de estado o con variables de cardinalidad alta producen espacios de estados cuya enumeración completa se vuelve computacionalmente costosa.

2. **Capa de representación exclusivamente.** El trabajo no modifica los algoritmos de inferencia exacta ni los compiladores de circuitos lógicos utilizados internamente por ProbLog. El efecto de las Disyunciones Anotadas sobre el tamaño de los circuitos compilados es un resultado emergente del compilador subyacente, no una optimización introducida por esta extensión.

3. **Convención de declaración de fluentes.** Todos los valores posibles de un fluente multivaluado deben declararse bajo el mismo nombre de predicado. Esta convención permite que el sistema identifique automáticamente los grupos mutuamente excluyentes, pero impide representar un fluente cuyos valores se distribuyan entre predicados distintos.

4. **Independencia entre variables de estado.** Las reglas de transición del programa deben respetar independencia entre las variables de estado. El sistema evalúa la probabilidad de transición de cada variable por separado y combina los resultados asumiendo que los factores son estocásticamente independientes. Si las reglas del usuario introducen dependencias entre variables que violan este supuesto, el sistema no lo detecta ni emite advertencias.

5. **Dominios de evaluación.** Las pruebas se ejecutan sobre dominios de grid discretos de dimensión reducida, seleccionados por la disponibilidad de soluciones de referencia en la literatura. Los resultados no se extrapolan a dominios de gran escala ni a entornos físicos en tiempo real.

Una vez delimitado el problema, los objetivos y los alcances de este trabajo, el siguiente capítulo analiza el estado del arte y trabajos que sustentan la propuesta, revisando los enfoques existentes para la representación factorizada de MDPs y el papel de la Programación Lógica Probabilística en la toma de decisiones secuenciales.

2. Estado del Arte

El presente capítulo rastrea la evolución de las tres líneas de investigación cuya intersección define el problema que esta tesina aborda: (1) la representación factorizada de los MDPs y el problema de las variables de estado; (2) el uso de la Programación Lógica Probabilística (PLP) como lenguaje de especificación para la toma de decisiones secuenciales; y (3) los mecanismos de exclusión mutua y compilación de conocimiento para la inferencia eficiente. El recorrido muestra cómo cada uno de ellos ha aportado los elementos necesarios para fundamentar la extensión propuesta.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

A medida que los dominios de planificación crecían en complejidad, quedó claro que ningún método que operara sobre estados individuales podría escalar ya que el tamaño del espacio de estados crece exponencialmente con el número de variables, y los algoritmos clásicos de programación dinámica requieren iterar sobre cada estado [3]. Este crecimiento fue reconocido por la como el obstáculo central que debía resolverse para hacer los MDPs aplicables a problemas reales, y motivó una línea de investigación dedicada a explotar la estructura interna del dominio para eludir la enumeración explícita.

Con este objetivo, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [12]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [6]. Los ADDs, al ser grafos dirigidos acíclicos, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Tanto SPI como SPUDD operan exclusivamente sobre variables booleanas debido a la descomposición de Shannon que subyace a ambas estructuras. Cada nodo interno ramifica sobre una variable binaria [6]. Cuando el dominio contiene variables multivaluadas, el modelador debe descomponerlas en múltiples variables binarias. Hoey *et al.* reconocen esta limitación al señalar que una variable ternaria debe expandirse en dos binarias, lo que aumenta el espacio de estados por un factor de 4/3 por variable e introduce estados semánticamente imposibles [6]. Esta binarización forzada constituye precisamente la limitación que se busca corregir en MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [13] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, RDDL adopta un DBN estratificado por niveles como representación semántica, donde cada variable de un paso temporal sólo puede depender de variables de niveles anteriores [13]. Esta estructuración acíclica impide modelar dependencias cíclicas entre predicados aterrizados, como las que surgen en dominios con relaciones simétricas o transitivas; MDP-ProbLog resuelve este problema al heredar de Prolog la resolución SLD, capaz de manejar definiciones recursivas sobre el programa aterrizado [4].

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La comunidad de MDPs factorizados desarrolló representaciones cada vez más eficientes del espacio de estados. Años más tarde, la Programación Lógica Probabilística propuso lenguajes relacionales capaces de razonar bajo incertidumbre [14]. La pregunta que eventualmente conectó ambas líneas de investigación fue si los motores de inferencia exacta de ProbLog podían adaptarse para resolver, en lugar de consultas estáticas, problemas de decisión secuencial en los que el agente debe optimizar su comportamiento a lo largo del tiempo.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [15]. Este sistema añadió al lenguaje la capacidad de modelar problemas de decisión episódicos mediante variables de decisión y atributos de utilidad, permitiendo encontrar la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado exclusivamente para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo. De esta forma, estableció la prueba de concepto de que ProbLog era extensible hacia la optimización bajo incertidumbre, aunque la resolución de MDPs requería un mecanismo adicional para propagar valor a través de horizontes de tiempo arbitrarios.

Como derivado de este trabajo, Bueno *et al.* propusieron MDP-ProbLog, que extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [4]. El solver implementa iteración de valor sobre el circuito compilado mediante Weighted Model Counting (WMC), demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un backup de Bellman. Esta equivalencia garantiza que la evaluación sobre el circuito produce exactamente las probabilidades de transición requeridas, permitiendo que el solver converja a políticas óptimas sin enumerar el espacio de estados. Una ventaja distintiva frente a lenguajes como RDDL es su capacidad para representar dependencias cíclicas entre predicados aterrizados gracias a la resolución SLD de Prolog [4].

2.3. Disyunciones Anotadas como mecanismo de exclusión mutua

En paralelo al desarrollo de los MDPs factorizados y de MDP-ProbLog, una línea independiente de investigación dentro de la Programación Lógica Probabilística se ocupaba de un problema distinto: cómo representar eventos con múltiples resultados mutuamente excluyentes sin imponer restricciones externas al programa. Vennekens, Verbaeten y Bruynooghe respondieron a este problema con los *Logic Programs with Annotated Disjunctions* (LPADs) [11], demostrando que es posible codificar la exclusión mutua de manera nativa en la cabeza de las reglas lógicas. Esta propiedad elimina la necesidad de restricciones manuales y evita la generación de estados semánticamente imposibles, dos problemas que los enfoques booleanos independientes trasladan al modelador.

En trabajo posterior, Vennekens, Denecker y Bruynooghe formalizaron una interpretación causal de las ADs mediante *CP-Logic* [16], demostrando que el cuerpo de cada regla representa una causa y los disyuntos en la cabeza sus posibles efectos, cada uno con una probabilidad de ocurrencia. Este alineamiento es directo con la dinámica estocástica de un MDP: dado un estado y una acción, el sistema transita hacia exactamente uno de varios estados siguientes posibles. Los autores demostraron además la equivalencia expresiva entre los LPADs y el Independent Choice Logic de Poole [17], consolidando a las ADs como el mecanismo más natural para representar efectos indeterminados en programas lógicos probabilísticos.

2.4. Compilación de conocimiento para inferencia eficiente

Para que la inferencia probabilística sobre programas con átomos de decisión sea computacionalmente tratable, sistemas como ProbLog emplean un flujo de compilación de conocimiento. El sistema transforma el programa lógico aterrizado en una fórmula proposicional que, a su vez, se compila en un circuito lógico. Sobre este circuito se ejecuta el Conteo de Modelos Ponderados (WMC). Darwiche y Marquis establecieron la taxonomía formal de los lenguajes de compilación e identificaron que estructuras como las d-DNNFs (*Deterministic Decomposable Negation Normal Forms*) permiten realizar inferencia exacta en tiempo lineal respecto al tamaño del circuito compilado [18]. Posteriormente, Darwiche introdujo los Diagramas de Decisión Sentencial (SDDs), una representación canónica que generaliza los clásicos BDDs y resulta estrictamente más sucinta [19].

Un avance directamente relevante para este trabajo es el algoritmo de diferenciación de circuitos propuesto por Darwiche [20], que permite calcular todas las derivadas parciales de un circuito d-DNNF en dos pasadas lineales sobre el mismo. Esta técnica amortiza el costo de inferencia cuando se requieren múltiples evaluaciones simultáneas, como ocurre durante la iteración de valor en MDP-ProbLog, donde cada par estado-acción demanda calcular las probabilidades de transición de todos los fluentes del siguiente estado. ProbLog2 integra estas estructuras como backends de compilación, ofreciendo flexibilidad para seleccionar entre d-DNNFs y SDDs según las compensaciones de espacio y tiempo del dominio [21]. Cualquier extensión a la capa de representación, como la integración de fluentes multivaluados propuesta, debe preservar la compatibilidad con este flujo de compilación para mantener las garantías de inferencia exacta.

2.5. Análisis Comparativo

La Tabla 4 compara los ocho enfoques revisados en este capítulo, agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, la exactitud de la inferencia, la capacidad relacional del lenguaje, y el mecanismo de exclusión mutua.

Tabla 4: Comparativa de los enfoques del estado del arte.

Trabajo	**Variables de estado**	**Inferencia**	**Expresividad relacional**	**Exclusión mutua**
MDPs factorizados proposicionales				
Boutilier *et al.* [12]	Booleanas	Exacta	Proposicional	Manual
Boutilier, Dean y Hanks [5]	Booleanas	Exacta	Proposicional	Manual
Hoey *et al.* [6]	Booleanas	Exacta	Proposicional	Manual
Lenguajes de planificación				
Sanner [13]	Multivaluadas / continuas	Aprox.	Relacional	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [14]	Booleanas	Exacta	Relacional	No aplica
Fierens *et al.* [22]	Booleanas	Exacta	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [16]	Multivaluadas	Exacta	Relacional	Nativa
MDP-ProbLog				
Bueno *et al.* [4]	Booleanas	Exacta	Relacional	Manual
Propuesta	**Multivaluadas**	**Exacta**	**Relacional**	**Nativa**

Como se observa, la propuesta reúne simultáneamente las cuatro capacidades: variables de estado multivaluadas, inferencia exacta, expresividad relacional y exclusión mutua nativa.

2.6. Conclusión del Estado del Arte

Los MDPs factorizados demostraron que la planificación a escala es viable cuando los algoritmos explotan las independencias del dominio [12, 6]. Posteriormente, MDP-ProbLog combinó la expresividad relacional con la inferencia exacta para resolver MDPs de horizonte infinito [4]. Por su parte, los LPADs y CP-Logic formalizaron la exclusión mutua nativa como una propiedad semántica de los programas lógicos, la cual se alinea perfectamente con la dinámica de las transiciones estocásticas [11]. Finalmente, la compilación de conocimiento proporcionó las estructuras algorítmicas sobre las que este mecanismo evalúa probabilidades en tiempo lineal [18, 20].

La revisión de la literatura evidencia que la programación lógica ofrece herramientas probadas para el modelado multivaluado mediante Disyunciones Anotadas que MDP-ProbLog desaprovecha y cuya corrección conlleva una intervención en la arquitectura del sistema.

Dado que el espacio de estados deja de representarse como una estructura de factores exclusivamente binarios, el motor de resolución necesita adoptar un esquema que permita a cada variable operar bajo su cardinalidad original. Asimismo, el algoritmo de Iteración de Valor debe contemplar el cálculo del valor esperado como una ramificación (n-aria) para así evaluar correctamente las transiciones dictadas por las Disyunciones Anotadas.

Sobre esta base, se diseña e implementa la extensión descrita. El siguiente capítulo establece los fundamentos teóricos detallados que sustentan esta integración, comenzando por las bases de la programación lógica y culminando en las propiedades de los circuitos compilados que hacen computacionalmente posible la toma de decisiones.

3. Marco Teórico

El capítulo anterior estableció que MDP-ProbLog vincula la programación lógica probabilística con la resolución de MDPs factorizados. El objetivo de este capítulo es brindar los fundamentos teóricos necesarios para comprender tanto la arquitectura original del framework como los formalismos lógicos que sustentan nuestra extensión.

Se comienza fijando nociones básicas de programación lógica (Sección 3.2). Sobre esa base, se introduce la incertidumbre mediante la semántica de distribución y el lenguaje ProbLog (Secciones 3.3 y 3.4). Las Disyunciones Anotadas, presentadas en la Sección 3.5, incorporan la exclusión mutua como propiedad nativa del formalismo. La Sección 3.6 describe el Conteo de Modelos Ponderados y la compilación de conocimiento, mecanismos que convierten las consultas probabilísticas en un cálculo tratable.

Posteriormente, se formalizan los Procesos de Decisión de Markov y su versión factorizada (Secciones 3.7 y 3.8). Esta formalización permite cuantificar el costo de representar variables categóricas mediante fluentes booleanos. Sobre esta base, se presenta el framework MDP-ProbLog (Sección 3.9). El Capítulo 4 describe la extensión de su arquitectura para soportar fluentes multivaluados.

3.1. Grid Estocástico

Para ejemplificar los conceptos presentados a lo largo de este capítulo, utilizaremos el siguiente dominio de referencia en el que el entorno consiste en un grid estocástico de 2x3 adaptado del dominio clásico de Russell y Norvig, cuyas celdas identificamos mediante pares (r, c) con $r \in \{1, 2\}$ y $c \in \{1, 2, 3\}$.

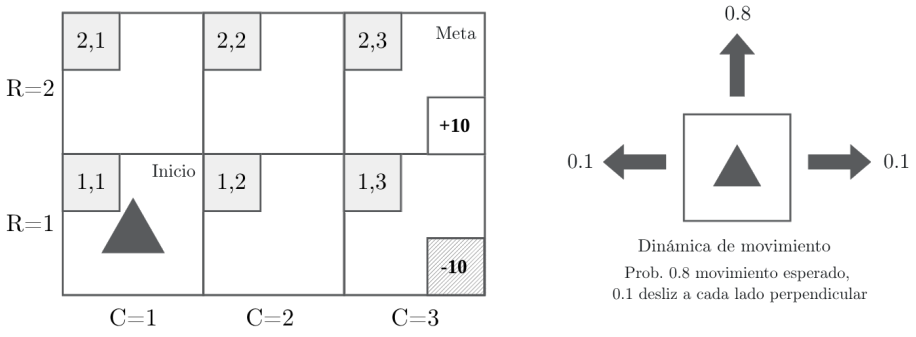


Figura 2: Grid de navegación 2x3 estocástico. Se incorporan transiciones probabilísticas.

La Figura 2 ilustra la configuración donde el agente inicia su recorrido en la celda (1,1) y busca alcanzar la meta para recibir una recompensa de +10. Se definen cuatro acciones

direccionales: norte, sur, este y oeste, donde cada transición incorpora incertidumbre.

Al ejecutar una acción, el agente avanza en la dirección deseada con una probabilidad del 0.8 y, con probabilidad de 0.1, se desliza hacia una dirección perpendicular. Toda acción actualiza la posición del agente, salvo cuando la celda de destino queda fuera del tablero, en ese caso, si el movimiento colisiona con un límite del entorno, el agente permanece en su posición actual.

3.2. Programación Lógica

Para representar formalmente sus estados y transiciones, MDP-ProbLog adopta el lenguaje de representación de la programación lógica. Esta sección establece la sintaxis del lenguaje, el proceso de resolución y la noción de instanciación que fundamentan la extensión probabilística de la sección siguiente.

Un programa lógico se construye a partir de hechos, reglas y consultas [23, 1]. Un hecho es una cláusula sin cuerpo que declara verdadero a un átomo. Un átomo es un predicado aplicado a sus argumentos. En el grid, el hecho `posicion(1,1).` afirma que el agente ocupa la celda $(1, 1)$. Una regla establece una implicación entre una cabeza y un cuerpo separados por el operador `:-`. La cabeza es verdadera solo cuando todos los literales del cuerpo son verdaderos.

La regla `meta_alcanzada :- posicion(2,3).` expresa que la condición de éxito se cumple si es que el agente ocupa la celda $(2, 3)$. Una consulta pregunta si un átomo es derivable a partir de un programa lógico definido. Si existe una prueba, el sistema devuelve la sustitución que lo hace verdadero, de lo contrario, falla.

Un programa lógico definido es un conjunto finito de cláusulas de Horn en el que cada cláusula tiene exactamente un átomo en la cabeza [23]. Las cláusulas pueden incluir variables escritas con mayúscula inicial, las cuales se interpretan como universalmente cuantificadas sobre el dominio de objetos. La regla `posicion(R,C) :- celda(R,C).` expresa que (R, C) es una posición válida para todo par que satisfaga `celda`. Esta capacidad de generalización permite describir espacios de estados extensos con un número reducido de cláusulas.

Para responder una consulta, Prolog emplea un proceso llamado resolución SLD [1] el cual opera de forma descendente sobre la lista de literales pendientes de la consulta. Dado un literal, el sistema busca una cláusula del programa cuya cabeza pueda unificarse con él.

Este proceso de unificación consiste en encontrar una sustitución de variables que haga idénticos a dos términos. Si se tiene éxito se aplica una sustitución y los literales unificados se reemplazan por el cuerpo de la cláusula seleccionada. Este proceso continúa recursivamente hasta que la lista queda vacía, considerandose una prueba exitosa, o hasta que ninguna cláusula puede unificarse con el literal actual.

El conjunto de todos los átomos base que pueden derivarse mediante unificaciones y resoluciones exitosas constituye precisamente el *modelo mínimo de Herbrand* del programa. Este

modelo es el conjunto más pequeño de átomos base que satisface todas las cláusulas del programa [23]. Por ejemplo, para el programa formado por el hecho `posicion(2,3).` y la regla `meta_alcanzada :- posicion(2,3).`, el modelo mínimo es {`posicion(2,3)`, `meta_alcanzada`}. En cambio, si el único hecho dl programa fuera `posicion(1,1).`, la regla no podría activarse y el modelo mínimo quedaría reducidoy a {`posicion(1,1)`}. De este modo, el modelo mínimo de Herbrand representa el conjunto exacto de consecuencias lógicas derivables del programa a través del proceso de resolución SLD.

Cuando un programa contiene variables, su instanciación produce el conjunto de todas las cláusulas base obtenidas al sustituir cada variable por una constante [1]. Este proceso se denomina *grounding* o instanciación y produce un programa aterrizado el cual contiene únicamente átomos base sin incluir variables. Un ejemplo de esto e la regla `celda(R,C) :- posicion(R,C).`, con $R \in \{1, 2\}$ y $C \in \{1, 2, 3\}$, produce en su instanciación seis cláusulas base, una para cada celda del dominio. Es así como el espacio de estados sobre el que opera el planificador se construye a partir de átomos base, aspecto que se retoma en la Sección 3.9. La sección siguiente introduce la incertidumbre extendiendo los hechos del programa con probabilidades.

3.3. Programación Lógica Probabilística

La Programación Lógica Probabilística (PLP) extiende el lenguaje de la sección anterior con una noción formal de incertidumbre. Para el lenguaje MDP-ProbLog, la PLP es la base sobre la que se calcula toda probabilidad de transición entre estados. Antes de describir cómo se realiza este cálculo, es necesario explicar qué significa exactamente que un programa lógico tenga una distribución de probabilidad asociada.

3.3.1. Semántica de distribución

Sato [24] observó que si se asigna una probabilidad independiente a cada hecho de un programa lógico, las reglas del programa propagan esa incertidumbre, induciendo una distribución sobre todos los modelos mínimos posibles. Esta idea, llamada *semántica de distribución*, es el fundamento común de la mayoría de los lenguajes de PLP modernos, incluido ProbLog.

Formalmente, un programa se representa como una tupla $DB = F \cup R$, donde $F = \{f_1, \ldots, f_k\}$ es el conjunto de hechos probabilísticos, cada uno asociado a una probabilidad $\theta_i \in [0, 1]$, y R es el conjunto de reglas determinísticas. Se asume que los hechos en F son estocásticamente independientes.

Una *realización total* L es una asignación completa de valores de verdad a todos los hechos de F, de modo que cada hecho f_i puede ser verdadero o falso. Por combinatoria, es fácil visualizar que, dado que existen k hechos probabilísticos y cada uno puede adoptar independientemente uno de dos valores de verdad, se tienen exactamente 2^k realizaciones totales posibles.

Es importante destacar que, para cada una de estas 2^k realizaciones L, su probabilidad se calcula multiplicando las probabilidades de los hechos que se consideran verdaderos por las probabilidades complementarias de aquellos que se consideran falsos:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización total L determina un subconjunto concreto de hechos verdaderos que, al combinarse con las reglas determinísticas R, produce un programa lógico definido $L \cup R$ cuyo modelo mínimo de Herbrand es único, denotado $M_{DB}(L)$.

Por consiguiente, la distribución de probabilidad definida sobre las realizaciones induce naturalmente una distribución de probabilidad sobre los modelos mínimos. Cuando el conjunto F de hechos probabilísticos es finito, Sato demostró que toda la masa de probabilidad del programa se concentra exactamente en estos modelos derivados. La probabilidad de éxito de una consulta q se define entonces como la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Para aterrizar estos conceptos en el grid, considérese una situación donde el agente esté en la celda $(1, 1)$ ejecutando la acción `norte`. La dinámica establece una probabilidad de 80 % de avanzar hacia $(2, 1)$, mientras que el 20 % restante se reparte en desplazamientos ortogonales no deseados.

Dado que las herramientas para modelar transiciones con multiples resultados mutuamente excluyentes se introducirán más adelante, este primer acercamiento agrupa cualquier desvío en un único hecho probabilístico de fallo general. El Listado 1 muestra este fragmento inicial.

```
% Probabilidad agrupada de cualquier desvio ortogonal
0.2::fallo_norte.

% Consecuencias de ir hacia el norte estando en la pos(1,1)
pos(2,1) :- \+fallo_norte.
pos(1,1) :- fallo_norte.

% Consulta
query(pos(2,1)).
```

Código 1: Programa probabilístico. Se agrupan los desvíos en un único fallo.

Este programa define dos mundos posibles, correspondientes a las dos realizaciones del único hecho probabilístico. La Tabla 5 detalla ambas realizaciones con su probabilidad y el modelo mínimo resultante.

Tabla 5: Mundos posibles inducidos por la semántica de distribución sobre el Código 1.

`fallo_norte`	P(mundo)	`pos(2,1)`	Modelo mínimo
falso	0.8	verdadero	{`pos(2,1)`}
verdadero	0.2	falso	{`fallo_norte`, `pos(1,1)`}

La probabilidad de éxito de la consulta es la suma de las probabilidades de los mundos donde es verdadera, es decir, $P(\texttt{pos(2,1)}) = 0.8$. Este es exactamente el valor esperado y muestra cómo la incertidumbre del hecho probabilístico se propaga de manera determinista, a través de las reglas, hasta la consulta.

La ecuación 2 suma sobre 2^k realizaciones, donde k es el número de hechos probabilísticos del programa. Esta suma es de complejidad #P-hard en el caso general [22], por lo que no existe algoritmo polinomial conocido que la evalúe directamente para programas arbitrarios. Esta barrera computacional justifica la compilación de conocimiento como mecanismo de reducción de costo, descrita en la Sección 3.6.

Cabe aclarar que la semántica de distribución requiere un lenguaje concreto que la haga operacional. La sección siguiente presenta ProbLog, el lenguaje que implementa esta semántica.

3.4. Lenguaje ProbLog

ProbLog [1, 14] implementa la semántica de distribución con una extensión sintáctica mínima sobre Prolog. La extensión consiste en el *hecho probabilístico*, denotado `p::f.`, que declara que el átomo `f` es verdadero con probabilidad p de forma independiente a los demás hechos. Los átomos, las reglas, las consultas y la resolución SLD conservan el significado establecido en la Sección 3.2.

El motor de inferencia de ProbLog transforma un programa en una respuesta probabilística mediante cuatro etapas [22]. Primero, el programa se instancia reemplazando variables por constantes del dominio. Segundo, la fórmula resultante se traduce a una fórmula booleana ponderada. Tercero, esta fórmula se compila a un circuito lógico. Por último, la probabilidad de la consulta se obtiene evaluando el circuito mediante Conteo de Modelos Ponderados. Las etapas de compilación y evaluación recibirán su explicación detallada en la Sección 3.6.

Usando esta sintaxis, el Código 1 puede refinarse para representar con mayor fidelidad la dinámica del grid. El Código 2 expresa la transición completa para la acción norte desde la celda $(1, 1)$ como tres hechos probabilísticos seguidos de las reglas que determinan la celda destino en cada caso.

```
% Hechos probabilisticos independientes
0.8::exito_norte.
0.1::desliz_oeste.
0.1::desliz_este.
```

```
% Reglas de transicion
pos(2,1) :- exito_norte.
pos(1,1) :- desliz_oeste. % Choca y permanece en su lugar
pos(1,2) :- desliz_este.
```

Código 2: Transición para la acción norte desde $(1, 1)$ con hechos probabilísticos independientes.

En el ejemplo, Los tres hechos `exito_norte`, `desliz_oeste` y `desliz_este` se declaran como estocásticamente independientes. Sin embargo, las características del grid exigen que estos eventos sean mutuamente excluyentes. En cada transición, el agente puede avanzar hacia exactamente una celda. Al tratarlos como independientes, la semántica de distribución genera $2^3 = 8$ mundos posibles. Uno de esos mundos asigna valor verdadero a los tres hechos simultáneamente, con probabilidad $0.8 \times 0.1 \times 0.1 = 0.008$. Es así como se observa que existe un modelo mínimo donde se permiten las tres posiciones a la vez.

Para descartar estos mundos imposibles, el modelador debe agregar restricciones de integridad adicionales con la consecuencia de expandir el espacio de evaluación y oscurecer la descripción del modelo. La sección siguiente introduce el mecanismo que elimina este problema por construcción semántica.

3.5. Disyunciones Anotadas

Se ha comprobado que la representación de hechos probabilísticos independientes suele generar problemas cuando el modelador necesita representar una elección entre n resultados mutuamente excluyentes. Los 2^n mundos posibles incluyen combinaciones donde múltiples resultados son verdaderos simultáneamente, sin correspondencia con la dinámica real del sistema. Las Disyunciones Anotadas (*Annotated Disjunctions*, ADs), introducidas por Vennekens, Verbaeten y Bruynooghe [11], incorporan la exclusión mutua en la sintaxis de la cláusula. Esta modificación elimina los mundos espurios por construcción semántica.

Una Disyunción Anotada es una cláusula de la forma

$$(h_1 : p_1) \vee (h_2 : p_2) \vee \cdots \vee (h_n : p_n) \leftarrow b_1, \ldots, b_m \tag{3}$$

donde $h_1, \ldots, h_n$ son átomos, $p_1, \ldots, p_n$ son probabilidades con $\sum_{i=1}^{n} p_i \leq 1$, y $b_1, \ldots, b_m$ es el cuerpo de la cláusula [11]. Cuando la suma es estrictamente menor a uno, existe una probabilidad residual $1 - \sum_i p_i$ de que ningún head sea derivado. En transiciones de estado de un MDP la suma es igual a uno. Esta condición garantiza que cada instancia de la cláusula produzca exactamente un resultado.

La semántica de las ADs se define mediante el concepto de selección. Una selección σ asigna a cada cláusula AD del programa exactamente uno de sus heads. La instancia P_σ es el programa lógico definido que resulta de reemplazar cada cláusula AD por el hecho $h_{\sigma(r)}$ correspondiente a la cláusula r. La probabilidad de la selección σ es [11]

$$C_\sigma = \prod_r p_{\sigma(r)} \tag{4}$$

donde el producto recorre todas las cláusulas AD del programa. La distribución sobre interpretaciones se obtiene sumando C_σ sobre todas las selecciones cuya instancia P_σ deriva la interpretación objetivo.

El Código 3 reescribe la transición norte del grid de la Sección 3.4 como una única Disyunción Anotada. Esta cláusula genera exactamente tres selecciones. La Tabla 6 muestra la distribución resultante.

```
0.8::pos(2,1) ; 0.1::pos(1,1) ; 0.1::pos(1,2).
```

Código 3: Transición norte desde $(1, 1)$ representada como Disyunción Anotada.

Tabla 6: Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible.

Selección σ	C_σ	Modelo mínimo
$\sigma \to$ `pos(2,1)`	0.8	{`pos(2,1)`}
$\sigma \to$ `pos(1,1)`	0.1	{`pos(1,1)`}
$\sigma \to$ `pos(1,2)`	0.1	{`pos(1,2)`}

En contraste con los ocho mundos de la Tabla 5, la AD genera tres mundos en los que exactamente un destino es verdaderom lo que permite que no exista ningún mundo con dos posiciones verdaderas simultáneamente. Laselección garantiza que en cada mundo exactamente un head de la disyunción sea derivado.

Los Programas Lógicos con Disyunciones Anotadas (LPADs) son expresivamente equivalentes a la Lógica de Elección Independiente (ICL) de Poole [17] y a CP-Logic [16]. Los tres formalismos definen la misma clase de distribuciones sobre interpretaciones de Herbrand. Sus diferencias residen en la notación empleada y en la interpretación causal que se asigna a las reglas.

ProbLog clasifica internamente cada Disyunción Anotada como un conjunto de nodos `choice` en un constructo interno llamado ClauseDB, donde cada nodo . Cada nodo `choice` almacena el identificador del grupo AD al que pertenece y el átomo head asociado.[22] La sección siguiente presenta el mecanismo de inferencia que transforma los programas con ADs en evaluacionesde probabilidad de costo lineal.

3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento

Aunque la selección consiga una distribución exacta sobre las interpretaciones de un programa, calcular la probabilidad de una consulta q bajo esta distribución sigue requiriendo sumar sobre todas las selecciones compatibles con q, por lo que esta suma presenta la misma complejidad #P-hard que la Ecuación 2 exhibe para los hechos independientes. El Conteo de Modelos Ponderados (*Weighted Model Counting*, WMC) convierte esta suma en una evaluación aritmética sobre una representación compacta del programa.

Primero, establezcamos que para poder aplicar WMC sobre un probgrama ProbLog, primero hay que convertirlo a una fórmula booleana ponderada donde se define una función de pesos w sobre los literales [22]. Para cada átomo probabilístico con probabilidad p, se asigna $w(l) = p$ al literal positivo y $w(\neg l) = 1 - p$ al negativo. Para los átomos derivados, ambos literales reciben peso 1. El peso de una interpretación ω es el producto de los pesos de todos sus literales:

$$W(\omega, w) = \prod_{l \in \omega} w(l) \tag{5}$$

El WMC de la fórmula φ bajo w es la suma de los pesos de todas sus interpretaciones satisfacibles:

$$\mathrm{WMC}(\varphi, w) = \sum_{\omega \models \varphi} W(\omega, w) \tag{6}$$

Fierens et al. demuestran que la probabilidad de evidencia de un programa ProbLog es igual al WMC de su fórmula ponderada [22]. Comparando las Ecuaciones 6 y 1, se observa que $W(\omega, w)$ coincide con $P(L \mid DB)$. Esta equivalencia transfiere el problema de inferencia probabilística al dominio aritmético de las fórmulas booleanas ponderadas.

El WMC de una fórmula arbitraria sigue siendo #P-duro. La compilación de conocimiento aborda este problema transformando la fórmula en una estructura que admite WMC en tiempo lineal [18]. Esta estructura es la Forma Normal de Negación Determinista y Descomponible (*d-DNNF*, por sus siglas en inglés), la cual parte de otras representaciones similares, pero que cumple con ciertas propiedades que serán descritas a continuación. Una Forma Normal de Negación (NNF) es un grafo dirigido acíclico cuyos nodos internos son conjunciones o disyunciones y cuyos nodos hoja son literales. Una NNF es descomponible si los hijos de cada nodo conjuntivo no comparten átomos entre sí. Una NNF es determinista si los hijos de cada nodo disyuntivo son lógicamente inconsistentes entre sí [18]. Una d-DNNF satisface ambas propiedades.

La descomponibilidad permite computar el WMC de cada nodo conjuntivo como el producto de los WMC de sus hijos. El determinismo permite computar el WMC de cada nodo disyunti-

vo como la suma de los WMC de sus hijos. La propiedad de suavidad exige que todos los hijos de un nodo disyuntivo mencionen el mismo conjunto de átomos. Con estas tres propiedades satisfechas, el recorrido ascendente sobre el circuito completo calcula WMC en $O(|C|)$, donde $|C|$ es el número de nodos del circuito [22, 18]. El circuito resultante, denominado circuito aritmético, puede compilarse una sola vez por programa y reutilizarse para evaluar todas las consultas posteriores sin recompilación.

Darwiche demostró que todas las derivadas parciales $\partial\,\mathrm{WMC}(\varphi, w)/\partial w(l)$ para cada literal l del circuito pueden calcularse simultáneamente mediante dos recorridos lineales [20]. El primero, ascendente, calcula el WMC y los valores intermedios de cada nodo. El segundo, descendente, propaga las contribuciones hacia los nodos hijos. La derivada $\partial\,\mathrm{WMC}/\partial w(l)$ es proporcional a la probabilidad marginal de l bajo la distribución del programa. Todas las probabilidades marginales se obtienen así en $O(|C|)$, sin necesidad de evaluaciones independientes del circuito.

ProbLog2 ofrece como alternativa al compilador d-DNNF el uso de Diagramas de Decisión Sentencial (SDDs), introducidos por Darwiche [19], los cuales son un subconjunto estricto de los d-DNNFs que imponen propiedades adicionales sobre la descomposición del espacio de variables. Un SDD con n variables y *treewidth* t tiene tamaño acotado por $O(n \cdot 2^t)$ [19]. Adicionalmente, los SDDs soportan combinación booleana en tiempo polinomial. Esta propiedad facilita la construcción incremental de circuitos cuando se añaden cláusulas al programa.

Durante la Iteración de Valor de MDP-ProbLog, cada respaldo de Bellman requiere evaluar $P(s' \mid s, a)$ para todos los posibles estados siguientes s' [4]. Estas evaluaciones corresponden a múltiples consultas WMC sobre el mismo circuito compilado. El algoritmo de diferenciación de Darwiche obtiene todas estas probabilidades en dos recorridos del circuito, independientemente del número de posibles estados siguientes. La compilación se realiza una sola vez y el circuito se reutiliza en cada iteración del algoritmo. La sección siguiente introduce los Procesos de Decisión de Markov, el marco sobre el cual MDP-ProbLog calcula políticas óptimas.

3.7. Procesos de Decisión de Markov

Los Procesos de Decisión de Markov (MDPs) formalizan la toma de decisiones secuencial bajo incertidumbre. Un agente observa el estado del entorno y selecciona una acción. La acción produce una recompensa y el entorno transita al siguiente estado según una distribución estocástica. El objetivo del agente es maximizar la suma de recompensas futuras descontadas. MDP-ProbLog utiliza este marco para calcular la política óptima de un agente en un entorno probabilístico.

Un MDP de horizonte infinito con factor de descuento es una tupla (S, A, T, R, γ), donde S es un conjunto finito de estados, A es un conjunto finito de acciones, $T : S \times A \times S \to [0, 1]$ es la función de transición estocástica con $\sum_{s' \in S} T(s, a, s') = 1$ para todo par (s, a), $R : S \times A \to \mathbb{R}$ es la función de recompensa inmediata y $\gamma \in [0, 1)$ es el factor de descuento [3]. El horizonte infinito con descuento optimiza sobre un futuro indefinido, donde las recompensas lejanas pesan menos que las inmediatas. La función de valor óptima $V^* : S \to \mathbb{R}$ satisface la ecuación

de optimalidad de Bellman [2]:

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s') \right] \tag{7}$$

Esta ecuación establece que el valor óptimo de un estado es la recompensa inmediata máxima más el retorno descontado esperado del estado siguiente. La política óptima $\pi^* : S \to A$ elige en cada estado la acción que maximiza el lado derecho de la Ecuación 7. En el grid de la Sección 3.1, la acción óptima en cada celda depende tanto de la recompensa en la meta como de la distancia que falta para alcanzarla bajo el factor de descuento γ.

La Iteración de Valor (IV) calcula V^* de forma iterativa [3]. El algoritmo inicializa $V_0(s) = 0$ para todo s y aplica repetidamente el operador de Bellman:

$$V_{k+1}(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V_k(s') \right] \tag{8}$$

El algoritmo converge cuando el residuo máximo $\max_{s \in S} |V_{k+1}(s) - V_k(s)|$ cae por debajo del umbral $\varepsilon(1-\gamma)/\gamma$ [3]. La Figura 3 muestra la política π^* y los valores V^* del grid de la Sección 3.1 calculados con $\gamma = 0.9$ y $\varepsilon = 0.1$. Los valores son mayores en las celdas próximas a la meta en $(2,3)$ y decrecen con la distancia, reflejando el efecto acumulado del descuento.

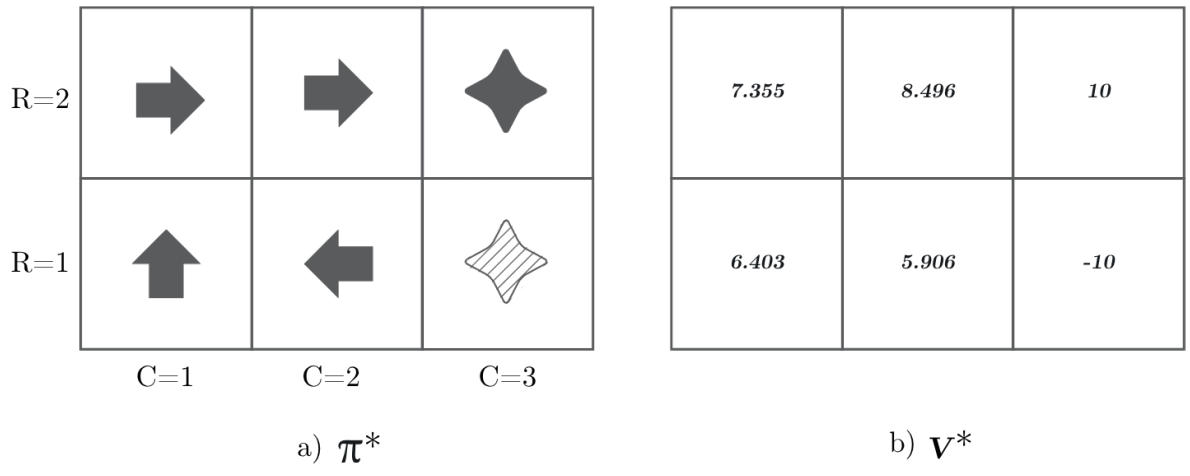


Figura 3: Política óptima: (a) una flecha indica la acción óptima $\pi^*(r,c)$; (b) muestra el valor $V^*(r,c)$ calculado por Iteración de Valor.

La Iteración de Valor opera sobre la enumeración explícita de S, por lo que la complejidad de cada respaldo es $O(|S|^2 \cdot |A|)$. A medida que el número de variables de estado crece, $|S|$ crece exponencialmente y el algoritmo deja de ser aplicable directamente. La sección siguiente examina cómo la representación factorizada aborda esta limitación.

3.8. MDPs Factorizados y Fluentes de Estado

Un MDP en el que el estado se describe mediante n variables tiene un espacio de estados de tamaño $\prod_{i=1}^{n} |\mathrm{dom}(X_i)|$. Para n variables binarias, $|S| = 2^n$. Para dominios con decenas de variables, la enumeración explícita de S resulta computacionalmente prohibitiva. Los MDPs Factorizados responden a este problema representando los estados y las transiciones mediante la estructura interna de las variables de estado [12].

En un MDP Factorizado, el estado s es una valuación de conjunto de variables $\mathbf{X} = (X_1, \ldots, X_n)$, donde cada X_i tiene dominio finito $\mathrm{dom}(X_i)$ [12]. La función de transición se factoriza de acuerdo con la estructura de una Red Bayesiana Dinámica (DBN). Para cada variable X_i, el valor siguiente X_i' depende únicamente de un subconjunto de variables padre $\mathrm{Pa}(X_i') \subseteq \mathbf{X}$:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P\big(X_i' \mid \mathrm{Pa}(X_i'),\, a\big) \tag{9}$$

Esta factorización expresa la dinámica del sistema en términos de dependencias locales. El número de parámetros necesarios crece linealmente con el número de variables en lugar de cuadráticamente con $|S|$.

En MDP-ProbLog, las variables del MDP Factorizado se denominan fluentes de estado. Un fluente de estado es una variable con un dominio discreto finito [4]. Un fluente booleano tiene dominio $\{0, 1\}$. Un fluente multivaluado tiene dominio $\{v_1, \ldots, v_k\}$ con $k \geq 2$. La sección siguiente describe cómo MDP-ProbLog integra el formalismo de los MDPs Factorizados con la programación lógica probabilística.

3.9. El framework MDP-ProbLog

MDP-ProbLog integra los MDPs Factorizados junto con el lenguaje ProbLog para resolver problemas de planificación probabilística de horizonte infinito. La idea principal se basa en la correspondencia entre la evaluación de un programa ProbLog bajo evidencia y el respaldo de Bellman sobre un estado del MDP [4]. En MDP-ProbLog, el usuario crea el modelo declarativamente en ProbLog y el solver transforma esa especificación en una política óptima mediante Iteración de Valor sobre el circuito compilado.

Un programa MDP-ProbLog extiende la sintaxis de ProbLog con tres categorías de predicados reservados [4]. El predicado `state_fluent/1` declara los fluentes de estado cuya valuación define un estado del MDP. El predicado `action/1` declara las acciones disponibles en cada

estado. El predicado `utility/2` encapsula la función de recompensa, asociando un átomo con un valor numérico.

La dinámica del MDP se especifica mediante un programa de dos instantes temporales. Los átomos etiquetados con el marcador $t = 0$ representan el estado actual y la acción ejecutada, a su vez, los átomos etiquetados con $t = 1$ representan el estado siguiente. Las reglas de transición relacionan el estado en $t = 0$ con el estado en $t = 1$ a través de hechos probabilísticos auxiliares que codifican la incertidumbre de las acciones.

El Código 4 enseña la estructura de un programa MDP-ProbLog para el grid de la Sección 3.1 con codificación booleana. Los tres fluentes booleanos `row2(0)`, `col2(0)` y `col3(0)` codifican la posición del agente usando tres bits, esquema que produce $2^3 = 8$ combinaciones, de las cuales solo seis son válidas, tal y como se describió en la Sección 3.8.

```
% Fluentes de estado: codificacion booleana de la posicion del
   agente
% (3 bits generan 2^3 = 8 combinaciones; solo 6 corresponden a
   celdas reales)
state_fluent(row2(0)).    % fila 2 si verdadero, fila 1 si falso
state_fluent(col2(0)).    % columna 2 si verdadero
state_fluent(col3(0)).    % columna 3 si verdadero

% Acciones
action(norte). action(sur). action(este). action(oeste).

% Incertidumbre estocastica para la accion norte
0.8::avance_n(0).   0.1::desvio_e_n(0).   0.1::desvio_o_n(0).

% Transicion: avance hacia la fila 2 con accion norte (regla
   representativa)
row2(1) :- norte(0), avance_n(0), \+ row2(0).
% ... (reglas de transicion para col2, col3 y las demas acciones)

% Recompensa: +10 al alcanzar la celda meta (fila 2, columna 3)
meta(1) :- row2(1), col3(1), \+ col2(1).
utility(meta(1), 10).
```

Código 4: Estructura de un programa MDP-ProbLog para el grid 2×3 con codificación booleana.

Bueno et al. demuestran que evaluar un programa MDP-ProbLog bajo evidencia (s, a) produce exactamente las probabilidades de transición $P(s' \mid s, a)$ requeridas por el respaldo de Bellman [4]. La evaluación se realiza mediante WMC sobre el circuito compilado. El resultado es equivalente al lado derecho de la Ecuación 7. Esta equivalencia garantiza que el algoritmo de IV de MDP-ProbLog calcula la misma política óptima que la IV clásica.

El solver de MDP-ProbLog ejecuta tres etapas [4]. En la etapa de preprocesamiento, se

resuelven las reglas intencionales y se construye el programa aumentado con los predicados de función de valor. En la etapa de compilación, el programa aterrizado se convierte a una fórmula booleana ponderada y se compila a un circuito d-DNNF. En la etapa de Iteración de Valor, el solver fija la evidencia (s, a) para cada par estado-acción, evalúa el circuito mediante WMC y actualiza V hasta convergencia.

La restricción booleana de la versión original reside en la etapa de preprocesamiento. Cada fluente de estado se inyecta como el hecho probabilístico `0.5::f(0)`, que presupone un dominio binario $\{0, 1\}$. Esta decisión impide representar directamente variables categóricas sin codificación binaria manual. La extensión propuesta interviene exclusivamente en esta capa de inyección. Para los fluentes multivaluados, sustituye los hechos probabilísticos independientes por Disyunciones Anotadas. El resto del pipeline permanece sin modificaciones.

4. Diseño e Implementación

El capítulo anterior estableció los formalismos que fundamentan la extensión los cuales son la Programación Lógica Probabilística como base de razonamiento, las Disyunciones Anotadas y el Conteo de Modelos Ponderados. El presente capítulo transforma estos conceptos en artefactos de software, comenzando por el análisis de la arquitectura original (Sección 4.1) y culminando en la demostración de retrocompatibilidad con los modelos booleanos existentes (Sección 4.8). Entre ambos, las Secciones 4.2 a 4.7 describen el esquema factorizado del espacio de estados, el clasificador automático de fluentes, el *pipeline* de preparación del MDP, la adaptación del algoritmo de Iteración de Valor, el evaluador de Darwiche y los módulos de soporte.

4.1. Análisis de la arquitectura original

El punto de partida de cualquier extensión es comprender la arquitectura que se extiende. MDP-ProbLog opera bajo un diseño modular que separa la inferencia probabilística, la representación del espacio de estados, la orquestación del MDP y la resolución numérica en módulos con responsabilidades delimitadas [4]. El análisis de esta arquitectura permite identificar dónde reside la restricción booleana y qué módulos requiere modificar la extensión.

4.1.1. Componentes del sistema original

La arquitectura original separa las responsabilidades del *framework* en cuatro módulos. Cada módulo tiene una función delimitada y las dependencias fluyen en una sola dirección. La Figura 4 muestra esta estructura; a continuación se describe la responsabilidad de cada módulo:

`engine.py` Implementa un adaptador sobre el motor interno de ProbLog (`DefaultEngine`). Encapsula la Base de Datos de Cláusulas (`ClauseDB`) y expone métodos para añadir hechos lógicos, ejecutar el *grounding* relevante y compilar circuitos evaluables para el Conteo de Modelos Ponderados.

`fluent.py` Define las abstracciones del estado del sistema. Las clases `Fluent` y `StateSpace` generan e indexan las configuraciones posibles del espacio de estados, asumiendo un dominio factorizado puramente booleano.

`mdp.py` Actúa como coordinador del *framework*. Interroga la `ClauseDB` buscando predicados reservados (`state_fluent/1`, `action/1`, `utility/2`), coordina la incorporación de hechos temporales y traduce las relaciones lógicas en una semántica de transiciones de Markov.

`value_iteration.py` Implementa el algoritmo de Iteración de Valor. Opera sobre índices enteros y distribuciones marginales para calcular la función de valor óptima mediante la ecuación de optimalidad de Bellman.

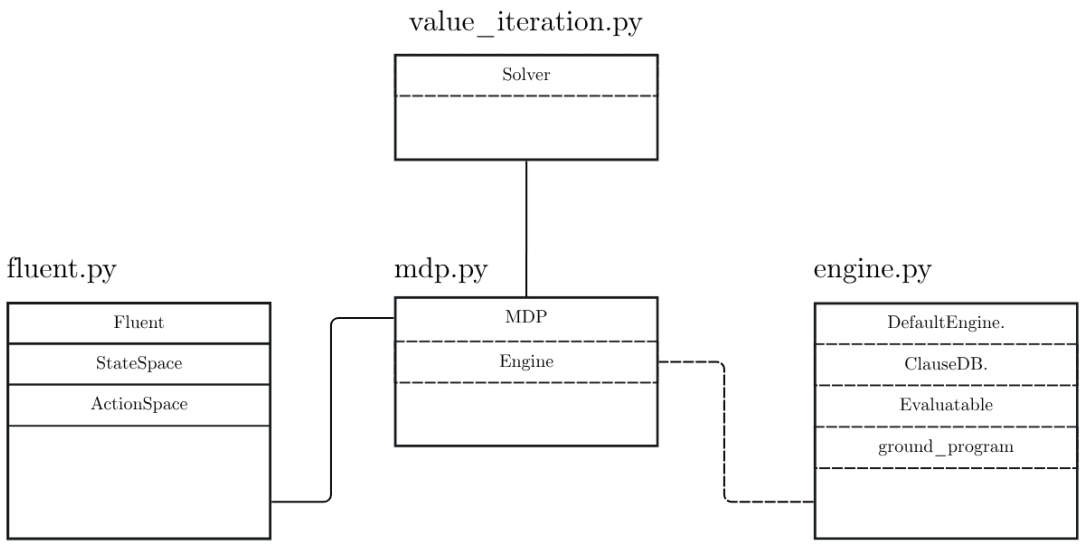


Figura 4: Diagrama de bloques de la arquitectura original del framework MDP-ProbLog.

El módulo `mdp.py` depende de `engine.py`, `fluent.py` y `value_iteration.py`. Los módulos `engine.py` y `fluent.py` son independientes entre sí. Esta organización delimita con claridad el alcance de la extensión propuesta. Los componentes que gestionan la estructura del espacio de estados y la orquestación del MDP constituyen los principales candidatos a modificación. Por su parte, el motor de inferencia encapsulado en `engine.py` permanece inalterado.

4.1.2. Flujo de ejecución

La interacción entre estos módulos sigue un flujo de cinco fases:

1. **Análisis y carga.** El módulo `engine.py` realiza el *parsing* del archivo Prolog proporcionado por el usuario y lo inyecta en la `ClauseDB`.

2. **Descubrimiento del dominio.** El coordinador `mdp.py` interroga la `ClauseDB` para identificar los fluentes de estado y las acciones declaradas, e instancia los iteradores correspondientes en `fluent.py`.

3. **Inyección temporal.** Para modelar la dinámica secuencial, `mdp.py` convierte las declaraciones abstractas en versiones con marca de tiempo ($t = 0$ para el estado actual, $t = 1$ para el siguiente estado). En esta fase se introducen hechos probabilísticos neutrales en $t = 0$ que serán reemplazados por evidencia determinista durante la evaluación de cada par estado-acción.

4. **Aterrizado y compilación.** Se ejecuta un *grounding* relevante respecto a las consultas de interés. Las reglas de transición se compilan en un circuito proposicional evaluable (SDD o d-DNNF, según el *backend* seleccionado).

5. **Resolución.** El módulo `value_iteration.py` recorre el espacio de estados, inyecta cada par estado-acción como evidencia en el motor y solicita las probabilidades marginales $P(s' \mid s, a)$ mediante WMC para actualizar la función de valor $V(s)$.

Dos de estas cinco fases asumen implícitamente un dominio booleano. La Fase 3 inyecta hechos probabilísticos independientes para cada fluente, presuponiendoque cada variable de estado tiene dominio $\{0, 1\}$. La Fase 5 recorre un espacio de estados construido como el producto cartesiano de factores binarios. La extensión modifica precisamente estas dos fases, junto con la estructura de datos que las sustenta.

4.1.3. Limitaciones en la gestión de fluentes

Como se estableció en el Capítulo 3, forzar fluentes categóricos en variables de Bernoulli degrada tanto el modelado lógico como la eficiencia computacional debido a la explosión de estados espurios.

El diseño original implementa la clase `StateSpace` como un iterador combinatorio plano que procesa las declaraciones `state_fluent/1` y generando mecánicamente el producto cartesiano completo del dominio binario sin tener en cuenta la noción de agrupamiento o exclusión mutua.

Lo anterior demuestra que la solución no requiere alterar el motor de inferencia probabilística de ProbLog ya que la compilación a circuitos lógicos y el WMC pueden permanecer intactos como mecanismo de evaluacón únicamente haciendo intervención en la intermedia donde las declaraciones lógicas se transforman en variables de estado evaluables para el *solver*.

Concretamente se pueden establecer tres puntos de extensión para integrar el soporte de fluentes multivaluados:

1. **Representación del espacio de estados.** Reemplazar `StateSpace` por una estructura factorizada que admita factores de cardinalidad arbitraria, eliminando la generación de combinaciones inválidas.

2. **Inyección de hechos en la `ClauseDB`.** Adaptar la Fase 3 del *pipeline* para que los fluentes multivaluados se representen como Disyunciones Anotadas en lugar de hechos probabilísticos independientes.

3. **Cálculo del valor esperado.** Reestructurar la evaluación del valor futuro en la Iteración de Valor para que recorra los factores del esquema en lugar de enumerar combinaciones binarias.

4.2. Esquema factorizado del espacio de estados

Una vez identificados los tres puntos de extensión, el primer componente a diseñar es la estructura de datos que reemplaza a `StateSpace`. Esta estructura debe cumplir dos condiciones: representar factores de cardinalidad arbitraria y proveer operaciones de indexación y decodificación que el resto del *framework* pueda invocar de forma uniforme. La clase `FluentSchema` satisface ambas condiciones.

4.2.1. Modelo de datos

El `FluentSchema` organiza el espacio de estados como una secuencia ordenada de m factores independientes. Cada factor k es una lista de términos lógicos aterrizados que representan las opciones mutuamente excluyentes de esa variable de estado. La cardinalidad del factor k, denominada base b_k, es igual al número de opciones que contiene.

La clase mantiene tres estructuras internas:

1. **Lista de factores.** Una lista de listas de términos: el factor k contiene los b_k términos que representan sus opciones. La posición de cada término dentro de su factor determina su índice local $v_k \in [0, b_k - 1]$.

2. **Lista de bases.** Un arreglo de enteros $[b_1, b_2, \ldots, b_m]$ que registra la cardinalidad de cada factor. Los factores booleanos tienen base 2; los factores multivaluados tienen base $N \geq 2$.

3. **Lista aplanada.** La concatenación de todos los términos en orden de registro. Esta lista permite localizar el término completo a partir del índice global sin recorrer todos los factores.

Dos categorías de factor se distinguen semánticamente. Un **factor booleano** tiene base 2 y sus dos términos corresponden a los valores $\{0, 1\}$ de un fluente de Bernoulli. Un **factor multivaluado** tiene base $N \geq 2$ y sus N términos son los valores de una variable categórica,

codificados internamente en representación *one-hot* sobre las opciones del grupo. Esta distinción determina cómo se inyectan los hechos durante el *pipeline* de preparación del MDP: los factores booleanos se representan como hechos probabilísticos independientes, mientras que los factores multivaluados se representan como Disyunciones Anotadas.

4.2.2. Codificación *mixed-radix*

Con los factores registrados, el `FluentSchema` asigna a cada configuración del espacio de estados un índice entero $I \in [0, |\mathcal{S}|-1]$ mediante codificación de base mixta (*mixed-radix*). Esta codificación es la generalización natural de la codificación binaria: donde la representación booleana utiliza base 2 uniforme, la codificación *mixed-radix* admite una base distinta por posición.

El tamaño total del espacio de estados es el producto de todas las bases:

$$|\mathcal{S}| = \prod_{k=1}^{m} b_k \tag{10}$$

Para mapear una configuración a un índice, se calcula un *stride* S_k por factor, definido como el producto de las bases de todos los factores que lo preceden:

$$S_k = \prod_{i=1}^{k-1} b_i \quad \text{con } S_1 = 1 \tag{11}$$

Dada una configuración donde el factor k toma el valor local $v_k \in [0, b_k - 1]$, el índice global se obtiene como:

$$I = \sum_{k=1}^{m} v_k \times S_k \tag{12}$$

La decodificación recupera la valuación a partir del índice extrayendo sucesivamente el residuo módulo la base de cada factor:

$$v_k = I \text{ mód } b_k, \quad I \leftarrow \left\lfloor \frac{I}{b_k} \right\rfloor \tag{13}$$

Ambas operaciones recorren los m factores en tiempo $O(m)$. La codificación garantiza que cada índice $I \in [0, |\mathcal{S}| - 1]$ corresponde a exactamente una combinación semánticamente válida. Los estados espurios del diseño original no pueden generarse porque no existe un índice que los represente.

4.2.3. Ejemplo mixed-radix

Para ilustrar el esquema, considérese un dominio con dos variables de estado: el **color** de un objeto, con valores en $\{\texttt{rojo}, \texttt{verde}, \texttt{azul}\}$, y la **velocidad** del sistema, con valores en $\{\texttt{lenta}, \texttt{rápida}\}$. El `FluentSchema` registra dos factores: el primero de base 3 (color) y el segundo de base 2 (velocidad).

El tamaño del espacio es $|\mathcal{S}| = 3 \times 2 = 6$, frente a los $2^5 = 32$ estados que generaría la codificación booleana equivalente con cinco fluentes independientes. Los *strides* son $S_1 = 1$ y $S_2 = 3$. La Tabla 7 muestra los seis estados resultantes.

Tabla 7: Codificación *mixed-radix* para el dominio color × velocidad.

Índice I	**color** (v_1)	**velocidad** (v_2)	**Cálculo**
0	`rojo` (0)	`lenta` (0)	$0{\times}1 + 0{\times}3 = 0$
1	`verde` (1)	`lenta` (0)	$1{\times}1 + 0{\times}3 = 1$
2	`azul` (2)	`lenta` (0)	$2{\times}1 + 0{\times}3 = 2$
3	`rojo` (0)	`rápida` (1)	$0{\times}1 + 1{\times}3 = 3$
4	`verde` (1)	`rápida` (1)	$1{\times}1 + 1{\times}3 = 4$
5	`azul` (2)	`rápida` (1)	$2{\times}1 + 1{\times}3 = 5$

Por ejemplo, el estado con `azul` y `rápida` se codifica como $I = 2 \times 1 + 1 \times 3 = 5$. La decodificación parte de $I = 5$: el color se extrae como $5 \bmod 3 = 2$ (`azul`) y se actualiza $I \leftarrow \lfloor 5/3 \rfloor = 1$; la velocidad se extrae como $1 \bmod 2 = 1$ (`rápida`). El proceso termina al agotar los factores.

4.2.4. Operaciones relevantes

Dos operaciones del `FluentSchema` son invocadas recurrentemente por el resto del *framework*:

`get_factors_at(timestep)` Retorna la lista de factores con los términos marcados con la estampa de tiempo indicada. Esta operación es utilizada por `mdp.py` para obtener los factores del estado actual ($t = 0$) y del estado siguiente ($t = 1$) durante la preparación del *pipeline*.

El marcado temporal se realiza añadiendo el argumento entero como último argumento de cada término; por ejemplo, el término `color(rojo)` a tiempo $t = 1$ se convierte en `color(rojo, 1)`.

`get_local_index(k, term)` Dado el índice de factor k y un término lógico aterrizado, retorna su posición v_k dentro de ese factor. Esta operación es el núcleo del cálculo del valor esperado (Sección 4.5): durante el recorrido recursivo de los factores de transición, cada término devuelto por el evaluador debe traducirse a su índice local para acumular el índice global del estado sucesor.

El `FluentSchema` centraliza la lógica de representación del espacio de estados para que el resto del *framework* pueda acceder al espacio exclusivamente a través de estos procedimientos. La sección siguiente describe cómo el `FluentClassifier` construye este esquema automáticamente a partir de las declaraciones del programa lógico.

4.3. Clasificador automático de fluentes

El `FluentSchema` descrito en la sección anterior necesita ser construido antes de que el *framework* pueda ejecutar ninguna otra etapa. Su construcción requiere conocer, para cada fluente declarado, si debe representarse como factor booleano o como factor multivaluado. El predicado `state_fluent/1` no transporta esta información: declara un fluente, pero no su tipo. El módulo `FluentClassifier` obtiene esa información analizando la estructura interna de la `ClauseDB`, específicamente los nodos que las Disyunciones Anotadas del programa han dejado registrados. A partir de esta evidencia estructural, el clasificador decide el tipo de cada fluente y construye el esquema sin intervención del usuario.

4.3.1. Modos de declaración

El clasificador admite dos formas de declarar fluentes de estado. En el **modo implícito**, el usuario escribe `state_fluent(f(X))` y el *framework* infiere el tipo analizando si los valores que toma el argumento de `f` provienen de una Disyunción Anotada. En el **modo explícito**, el usuario escribe `state_fluent(f(X), multivalued)` o `state_fluent(f(X), bool)` e indica el tipo directamente. El modo explícito tiene prioridad sobre el implícito: si un fluente aparece en ambas formas, el *framework* emite una advertencia y conserva la declaración explícita.

El modo implícito es suficiente para la mayoría de los dominios. El modo explícito es necesario en dominios con familias de variables indexadas por entidad, como se discute al final de esta sección.

4.3.2. El índice invertido de Disyunciones Anotadas

La inferencia implícita depende de una estructura auxiliar llamada **índice invertido**. Este índice se construye recorriendo la tabla de nodos de la `ClauseDB` e identificando todos los nodos de tipo `choice`, que son las ramas estocásticas que ProbLog registra al procesar una Disyunción Anotada. Por cada nodo `choice`, el clasificador extrae el término asociado y registra cada uno de sus argumentos constantes como clave del índice, apuntando al identificador del grupo AD al que pertenece ese nodo.

El resultado es un diccionario de la forma:

$$\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$$

donde Σ es el conjunto de valores constantes y Γ es el conjunto de identificadores de grupos AD. La interpretación es directa: $\mathcal{I}(v)$ es el conjunto de grupos AD que han producido el valor v.

Para ilustrarlo, considérese un programa que incluye la siguiente regla de transición para el fluente `color`:

```
0.7::color(rojo,1); 0.2::color(verde,1); 0.1::color(azul,1)
    :- color(rojo,0), cambiar.
```

Al procesar esta AD, ProbLog registra tres nodos `choice` bajo un mismo grupo g_1: uno para `color(rojo,1)`, uno para `color(verde,1)` y uno para `color(azul,1)`. El clasificador extrae los argumentos constantes de cada término y construye:

$$\mathcal{I}(\texttt{rojo}) = \{g_1\}, \quad \mathcal{I}(\texttt{verde}) = \{g_1\}, \quad \mathcal{I}(\texttt{azul}) = \{g_1\}$$

Este índice queda disponible antes de que comience la clasificación.

4.3.3. Pipeline de clasificación

El método `classify()` ejecuta seis pasos secuenciales para construir el `FluentSchema`:

1. **Validación de declaraciones explícitas.** Se verifica que cada tag de tipo en las declaraciones `state_fluent/2` sea `bool` o `multivalued`. Los tags inválidos se acumulan y se lanzan como un único error compuesto (`FluentDeclarationError`) para que el usuario pueda corregirlos todos de una vez.

2. **Registro de fluentes explícitos.** Cada par `(término, tag)` se almacena en el registro explícito con su tipo ya conocido. Estos fluentes quedan excluidos de la inferencia implícita.

3. **Registro de fluentes implícitos.** Los fluentes declarados con `state_fluent/1` se agrupan por `(functor, aridad)`. Para cada grupo, se invoca `_infer_fluent_type()` (descrito a continuación) para determinar si el fluente es booleano o multivaluado.

4. **Fusión de registros.** Se construye un registro unificado combinando los resultados de los pasos 2 y 3. Las declaraciones explícitas sobrescriben las implícitas cuando coinciden en la misma clave.

5. **Separación por tipo.** El registro se recorre en orden alfabético. Los fluentes booleanos se añaden al esquema directamente mediante `schema.add_bool(term)`. Los fluentes multivaluados se acumulan por *functor* en un buffer temporal.

6. **Validación de cardinalidad.** Cada grupo multivaluado acumulado en el paso 5 se verifica: si contiene menos de dos opciones, se lanza un `FluentCardinalityError`, pues un factor de exclusión mutua requiere al menos dos alternativas. Los grupos válidos se registran mediante `schema.add_group(terms)`.

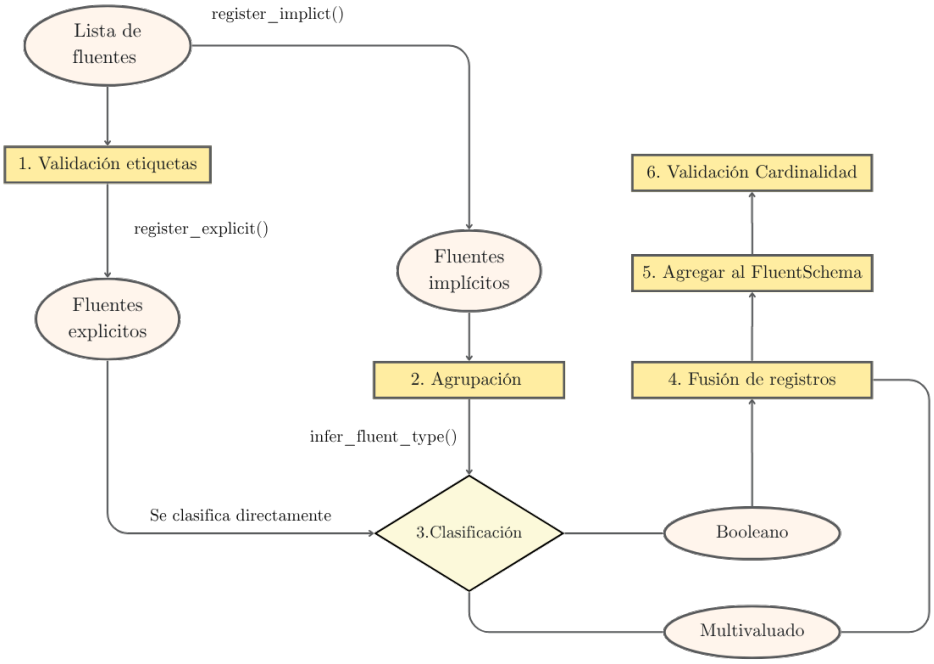


Figura 5: pipeline de clasificación. Se explica la ejecución del método `classify()`

4.3.4. Inferencia implícita de tipo

El paso 3 del *pipeline* delega la decisión de tipo en la función `_infer_fluent_type()`, cuya lógica se resume a determinar si es booleano comprobando si el fluente tiene aridad 0, esto debido a que los fluentes proposicionales no tienen argumentos, por lo tanto no pueden representar variables categóricas. Si el fluente tiene aridad ≥ 1, la función examina cada posición argumental p del término. Para la posición p, extrae el conjunto V_p de todos los valores que ese argumento toma en los términos aterrizados. Luego computa la intersección de los grupos AD asociados a cada valor en V_p mediante el índice invertido. Si esa intersección es no vacía, significa que todos los valores de la posición p provienen de un mismo grupo AD; en ese caso, el fluente es multivaluado. Si ninguna posición pasa esta prueba, el fluente es booleano.

El Algoritmo 1 formaliza esta lógica.

Algoritmo 1 Inferencia de tipo por intersección de orígenes estocásticos

Entrada: Conjunto de términos aterrizados G de aridad a; índice invertido $\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$.
Salida: Clasificación: `bool` o `multivalued`.

1: **if** $a = 0$ **then return** `bool`
2: **end if**
3: **for** $p \leftarrow 1$ **a** a **do**
4: $\quad V_p \leftarrow \{\arg_p(g) \mid g \in G\}$
5: $\quad C \leftarrow \bigcap_{v \in V_p} \mathcal{I}(v)$
6: $\quad$ **if** $C \neq \emptyset$ **then return** `multivalued`
7: $\quad$ **end if**
8: **end for**
return `bool`

Para verificar la intuición del algoritmo, aplíquese al fluente `color` del ejemplo anterior. Los términos aterrizados son `color(rojo)`, `color(verde)` y `color(azul)`, todos de aridad 1. En la posición 0, los valores son $V_0 = \{\texttt{rojo}, \texttt{verde}, \texttt{azul}\}$. La intersección es:

$$\mathcal{I}(\texttt{rojo}) \cap \mathcal{I}(\texttt{verde}) \cap \mathcal{I}(\texttt{azul}) = \{g_1\} \neq \emptyset$$

El algoritmo retorna `multivalued`. El fluente se registra como un factor de base 3 en el esquema.

El agrupamiento del paso 5 es correcto para variables categóricas globales, donde todos los términos con el mismo *functor* representan opciones de una única variable de estado. Para dominios donde múltiples entidades comparten predicado pero deben mantener independencia estocástica, el agrupamiento por *functor* puede producir factores incorrectamente fusionados; en esos casos, la declaración explícita con `state_fluent/2` permite definir cada factor de forma independiente, evitando que el clasificador los consolide.

Con el `FluentSchema` construido y validado, el *framework* dispone de la representación del espacio de estados necesaria para ejecutar el *pipeline* de preparación del MDP, que se describe en la sección siguiente.

4.4. Pipeline de preparación del MDP

Con el `FluentSchema` construido, el *framework* dispone de la descripción formal del espacio de estados. El siguiente problema es traducir ese esquema en una forma que el motor de inferencia pueda evaluar bajo condicionamiento. Esta traducción es responsabilidad del método `__prepare()` de la clase `MDP`, que ejecuta cinco fases antes de que comience la Iteración de Valor.

4.4.1. Fases del pipeline

1. **Clasificación.** Se instancia el `FluentClassifier`, que lee las declaraciones del programa y construye el `FluentSchema`. El resultado queda disponible para el resto de las fases.

2. **Inyección de hechos *dummy*.** Para cada factor del esquema se insertan hechos probabilísticos en la `ClauseDB` que representan el estado actual en $t = 0$. Estos hechos son *dummy*: sus probabilidades son valores neutrales que serán sustituidos en cada evaluación individual.

3. **Inyección de acciones.** Las acciones del modelo se inyectan como una Disyunción Anotada con distribución uniforme, garantizando exclusión mutua entre ellas.

4. **Aterrizado relevante.** Se ejecuta un *grounding* limitado a las consultas de interés: fluentes del siguiente estado ($t = 1$), utilidades, acciones y fluentes del estado actual ($t = 0$). Incluir los fluentes en $t = 0$ entre las consultas preserva sus identificadores en el circuito compilado, lo que permite localizarlos al sustituir evidencia.

5. **Compilación del circuito.** Los fluentes del siguiente estado y las utilidades se compilan en un único circuito d-DNNF o SDD. El circuito queda fijo y se reutiliza en todas las evaluaciones.

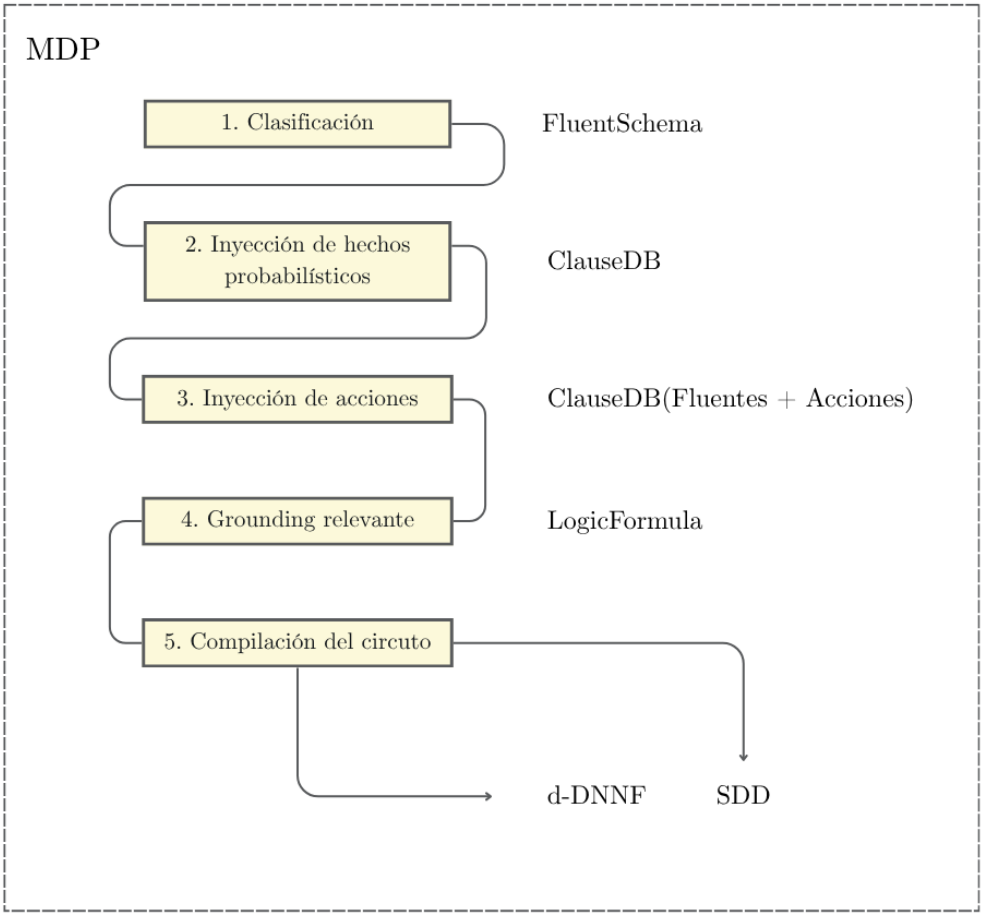


Figura 6: pipeline de preparación. Se explica la ejecución del método `__prepare()`

4.4.2. Hechos *dummy* y mecanismo de evidencia

La Fase 2 introduce el mecanismo técnico más relevante del *pipeline*: la inyección de hechos *dummy*. Para entender por qué es necesaria, conviene primero examinar cómo el *framework* implementa el condicionamiento por evidencia.

Durante la Iteración de Valor, el solver necesita evaluar $P(s' \mid s, a)$ para cada par estado-acción. Evaluar bajo un estado concreto significa fijar el valor de cada fluente en $t = 0$ a su valor en ese estado. ProbLog ofrece dos formas de condicionar: la primera propaga evidencia lógica a través del programa mediante el predicado `evidence/2`; la segunda sustituye directamente los pesos de los nodos probabilísticos en el circuito compilado, asignando peso 1 al literal verdadero y peso 0 al falso. El *framework* utiliza la segunda vía porque opera sobre el circuito compilado de forma directa y no requiere interpretar el programa de nuevo.

La sustitución de pesos impone una restricción: el nodo que se quiere condicionar debe existir como variable probabilística en el circuito. Un hecho determinista no tiene un peso que sustituir y no puede condicionarse por esta vía. Los hechos *dummy* resuelven esta restricción insertando cada fluente de estado en $t = 0$ como hecho probabilístico antes de la compilación, de modo que el compilador incluya cada fluente como un nodo con pesos en el circuito.

Los *dummies* se inyectan de forma distinta según el tipo de factor:

- **Factores booleanos.** Se inserta un hecho con probabilidad 0.5. Para el fluente `vivo`, la inyección produce `0.5::vivo(0)`.
- **Factores multivaluados.** Se inserta una Disyunción Anotada con distribución uniforme $1/N$ sobre las N opciones. Para el fluente `color` con tres valores, la inyección produce:

$$\tfrac{1}{3}\texttt{::color(rojo,0)}\ ;\ \tfrac{1}{3}\texttt{::color(verde,0)}\ ;\ \tfrac{1}{3}\texttt{::color(azul,0)}.$$

Las probabilidades neutrales de los *dummies* no afectan los resultados porque se reemplazan antes de cada consulta. Para condicionar sobre el estado `color` = `rojo`, el evaluador reemplaza los pesos del nodo `color(rojo,0)` por (1.0, 0.0) y los de los nodos restantes por (0.0, 1.0). El circuito calcula entonces $P(s' \mid \texttt{color=rojo}, a)$ exactamente, sin que sea necesario recompilarlo.

4.4.3. Corrección de la compilación doble

La versión original compilaba el circuito dos veces: una para los fluentes del siguiente estado ($t = 1$) y otra para las utilidades. Cada llamada al compilador sobrescribía el circuito almacenado en la clase `Engine`. En consecuencia, el mapa de nodos construido para los fluentes del siguiente estado quedaba invalidado por la segunda compilación, y las evaluaciones de transición operaban sobre el circuito de utilidades. Los valores de transición calculados eran incorrectos.

La extensión corrige este problema compilando ambos conjuntos de términos en una única invocación. Los fluentes del siguiente estado y las utilidades quedan mapeados al mismo circuito desde el inicio, y ese circuito permanece inalterado durante toda la Iteración de Valor.

4.4.4. Declaración de fluentes multivaluados

La extensión del *pipeline* habilita una sintaxis de modelado más directa para variables categóricas. El Código 5 muestra cómo se representaba la variable `color` con tres valores bajo el diseño original.

```
% Tres fluentes independientes
state_fluent(color_rojo).
state_fluent(color_verde).
state_fluent(color_azul).

% Transicion desde color=rojo bajo la accion "cambiar"
% (el modelador es responsable de garantizar exclusion mutua)
0.7::color_rojo(1) :- color_rojo(0), cambiar.
0.2::color_verde(1) :- color_rojo(0), cambiar.
0.1::color_azul(1) :- color_rojo(0), cambiar.
```

Código 5: Modelado booleano de una variable categórica con tres valores.

Con la codificación booleana, el espacio de estados para esta sola variable tiene $2^3 = 8$ combinaciones. El Código 6 muestra el modelo equivalente con la extensión.

```
% Una declaracion para las tres opciones
state_fluent(color(X), multivalued) :- val(X).
val(rojo). val(verde). val(azul).

% Transicion con AD: exclusion mutua y suma a 1 garantizadas
0.7::color(rojo,1) ; 0.2::color(verde,1) ; 0.1::color(azul,1)
    :- color(rojo,0), cambiar.
```

Código 6: Modelado de la misma variable con la extensión multivaluada.

La declaración multivaluada produce un único factor de base 3. El espacio de estados tiene exactamente 3 combinaciones, todas válidas. La Disyunción Anotada de la regla de transición garantiza por construcción que exactamente una opción es verdadera en cada estado sucesor y que la suma de probabilidades es 1. No se requieren restricciones adicionales de exclusión mutua.

4.5. Adaptación de la Iteración de Valor

La representación *mixed-radix* del espacio de estados necesita que el cálculo del valor esperado durante las actualizaciones de Bellman sea capaz de recorrer la estructura factorizada.

En el diseño original, ese cálculo enumeraba las 2^n combinaciones binarias del espacio. La nueva implementación recorre los factores del esquema de forma recursiva, explotando la independencia condicional entre ellos para evitar la enumeración explícita.

La conexión con el cálculo del valor esperado se establece así. La ecuación de Bellman requiere evaluar, para cada par (s, a):

$$\mathbb{E}_{s'}[V(s') \mid s, a] = \sum_{s'} P(s' \mid s, a)\, V(s') \tag{14}$$

Cuando el espacio de estados está factorizado en m variables independientes, la distribución de transición se descompone como:

$$P(s' \mid s, a) = \prod_{k=1}^{m} P(x'_k \mid s, a) \tag{15}$$

donde $P(x'_k \mid s, a)$ es la distribución marginal del factor k. El evaluador devuelve directamente estas distribuciones marginales para cada factor. La suma sobre todos los estados sucesores se convierte entonces en el producto cartesiano de estas distribuciones, que el algoritmo recorre factor a factor.

Descripción del algoritmo. El recorrido comienza en el factor $k = 0$ con el índice global acumulado $I = 0$ y la probabilidad conjunta $P = 1$. En cada llamada recursiva al nivel k, el algoritmo itera sobre las ramas del factor k, es decir, los pares (v, p) donde v es un valor posible del factor y p su probabilidad marginal. Para cada rama activa (con $p > \epsilon$), el algoritmo:

1. Obtiene el índice local v_k de ese valor dentro del factor k mediante `get_local_index`.
2. Actualiza el índice global acumulado con la contribución del factor: $I' = I + v_k \times S_k$.
3. Actualiza la probabilidad conjunta del camino: $P' = P \times p$.
4. Recurre al nivel $k + 1$ con I' y P'.

Cuando $k = m$ (todos los factores han sido procesados), el índice acumulado I identifica el estado sucesor s' en la codificación *mixed-radix*. El algoritmo retorna $P \times V[I]$, que es la contribución de ese camino al valor esperado. La suma de estas contribuciones sobre todos los caminos produce $\mathbb{E}[V(s') \mid s, a]$.

Algoritmo 2 Valor esperado del estado sucesor sobre el esquema factorizado

Entrada: Factor actual k; índice *mixed-radix* acumulado I; probabilidad conjunta del camino P.
Entrada: Distribuciones marginales por factor $\mathcal{F} = [\mathcal{F}_0, \ldots, \mathcal{F}_{m-1}]$, donde cada $\mathcal{F}_k$ es una lista de pares $(valor,\ prob)$.
Entrada: *Strides* $S = [S_0, \ldots, S_{m-1}]$; función de valor V; umbral de poda ϵ.
Salida: Valor esperado escalar $\mathbb{E}[V(s') \mid s, a]$ para el subárbol.

```
function ValorEsperado(k, I, P)
    if k = m then                                   ▷ Caso final. return P × V[I]
    end if
    acum ← 0
    for cada (v, p) ∈ F_k do
        if p > ε then                               ▷ Ignorar ramas con probabilidad despreciable.
            v_k ← ÍndiceLocal(k, v)                 ▷ Posición de v dentro del factor k.
            I' ← I + v_k × S_k                      ▷ Contribución de este factor al índice global.
            acum ← acum + ValorEsperado(k + 1, I', P × p)
        end if
    end forreturn acum
end function
```

Ejemplo de ejecución. Retómese el dominio con dos factores: **color** (base 3, strides $S_0 = 1$) y **velocidad** (base 2, stride $S_1 = 3$). Supóngase que el par estado-acción produce las siguientes distribuciones marginales: $\mathcal{F}_0 = [(\texttt{rojo},\ 0.7),\ (\texttt{verde},\ 0.2),\ (\texttt{azul},\ 0.1)]$ y $\mathcal{F}_1 = [(\texttt{lenta},\ 0.6),\ (\texttt{rápida},\ 0.4)]$.

La llamada inicial es ValorEsperado$(0,\ 0,\ 1.0)$. En el nivel $k = 0$ se exploran tres ramas; para la rama `rojo`: $v_0 = 0$, $I' = 0 + 0 \times 1 = 0$, se recurre a ValorEsperado$(1,\ 0,\ 0.7)$. En el nivel $k = 1$, rama `lenta`: $v_1 = 0$, $I' = 0 + 0 \times 3 = 0$, se retorna $0.7 \times 0.6 \times V[0]$. Rama `rápida`: $v_1 = 1$, $I' = 0 + 1 \times 3 = 3$, se retorna $0.7 \times 0.4 \times V[3]$. El proceso continúa para las ramas `verde` y `azul` del nivel $k = 0$. La suma de todas las contribuciones produce el valor esperado completo en exactamente $|\mathcal{F}_0| \times |\mathcal{F}_1| = 6$ hojas, que coincide con el número de estados válidos del espacio.

La poda por umbral ϵ reduce este recorrido en transiciones cuasi-deterministas. Si una sola rama por factor concentra casi toda la masa probabilística, el árbol efectivo tiene profundidad m y solo 1 hoja, frente a las $\prod_k |\mathcal{F}_k|$ hojas del caso general.

4.6. Evaluador de Darwiche

Al finalizar la Fase 5 del *pipeline* de preparación, el circuito está compilado y listo para ser evaluado. Durante la Iteración de Valor, el solver invoca ese circuito en cada par estado-acción para obtener las distribuciones de transición y las recompensas. Esta evaluación repetida es el cuello de botella computacional del *framework*. La presente sección describe el evaluador alternativo, basado en el algoritmo de diferenciación de circuitos de Darwiche (2003), que

reduce ese costo de forma asintótica.

4.6.1. El costo de la evaluación estándar

El evaluador estándar de ProbLog (`SimpleDDNNFEvaluator`) resuelve cada consulta realizando un recorrido completo del circuito, con costo $O(|C|)$ donde $|C|$ es el número de nodos. Cuando el motor necesita resolver Q consultas simultáneas, ejecuta Q recorridos independientes, con un costo total de $O(Q \times |C|)$ por par estado-acción.

Cada par estado-acción requiere una consulta por cada fluente del estado siguiente más una por cada término de utilidad. Para un modelo con m factores de estado y u utilidades, el valor de Q es $m + u$. Si el circuito tiene $|C|$ nodos y el espacio de búsqueda tiene $|\mathcal{S}| \times |\mathcal{A}|$ pares, el costo total de la Iteración de Valor es:

$$O(|\mathcal{S}| \times |\mathcal{A}| \times Q \times |C|) \tag{16}$$

El factor Q multiplica el trabajo por el número de consultas en cada evaluación. A medida que el número de factores de estado crece, Q crece con él, y el costo de cada iteración de Bellman aumenta proporcionalmente.

4.6.2. La idea central: derivadas como respuestas

La redundancia del evaluador estándar tiene una causa precisa. Cada uno de los Q recorridos recorre el circuito entero para extraer un único valor numérico, descartando la información estructural acumulada en los demás nodos. Si se pudiera preservar esa información entre consultas, bastaría con un recorrido único para responder todas las consultas de golpe.

Darwiche observó que el circuito compilado representa un polinomio multilineal $F(\lambda_1, \ldots, \lambda_n)$, donde cada λ_i es una variable indicadora asociada a un literal del circuito. Bajo esta interpretación, la probabilidad marginal de una consulta q satisface:

$$\Pr(q \mid e) = \frac{w_q \cdot \partial F / \partial \lambda_q}{F(e)} \tag{17}$$

donde w_q es el peso probabilístico del literal q y $F(e)$ es el valor del circuito bajo la evidencia e. Obtener las Q marginales equivale a calcular Q derivadas parciales del mismo polinomio. La ventaja es que todas las derivadas parciales pueden calcularse en exactamente dos recorridos lineales del circuito, independientemente de cuántas consultas haya.

4.6.3. El algoritmo de diferenciación en dos pasadas

El algoritmo asocia a cada nodo i del circuito dos cantidades: $\mathbf{val}(i)$, calculada de hojas a raíz, y $\mathbf{pd}(i)$, calculada de raíz a hojas. La primera es el valor de evaluación ascendente del nodo; la segunda es la derivada parcial de F respecto de ese valor, que cuantifica cuánto cambia la salida del circuito al modificar $\mathrm{val}(i)$.

Pasada ascendente. Se recorren los nodos en orden topológico desde las hojas hasta la raíz. El valor de cada nodo se calcula según su tipo:

- **Hoja** (literal λ_i con peso w_i): $\mathrm{val}(i) = w_i$. Si la evidencia activa hace el literal falso, $\mathrm{val}(i) = 0$.
- **Conjunción** (nodo AND): $\mathrm{val}(i) = \prod_{j \in \mathrm{hijos}(i)} \mathrm{val}(j)$.
- **Disyunción** (nodo OR): $\mathrm{val}(i) = \sum_{j \in \mathrm{hijos}(i)} \mathrm{val}(j)$.

Al terminar, $\mathrm{val}(\text{raíz}) = F(e)$, el WMC del circuito bajo la evidencia actual.

Pasada descendente. Se recorren los nodos en orden topológico inverso desde la raíz hasta las hojas. La raíz recibe $\mathrm{pd}(\text{raíz}) = 1$. Para cada nodo i y cada hijo j:

- **Padre OR**: $\mathrm{pd}(j) \mathrel{+}= \mathrm{pd}(i)$. La derivada del padre se transmite íntegra a cada hijo porque $\partial\,(\mathrm{val}(c_1) + \mathrm{val}(c_2))/\partial\,\mathrm{val}(c_j) = 1$.
- **Padre AND**: $\mathrm{pd}(j) \mathrel{+}= \mathrm{pd}(i) \times \prod_{k \in \mathrm{hijos}(i),\, k \neq j} \mathrm{val}(k)$. La derivada del padre se escala por el producto de los valores de los demás hijos porque $\partial\left(\prod_k \mathrm{val}(c_k)\right)/\partial\,\mathrm{val}(c_j) = \prod_{k \neq j} \mathrm{val}(c_k)$.

El acumulado $+=$ es necesario cuando un nodo tiene múltiples padres (circuitos tipo DAG): la regla de la cadena suma las contribuciones de cada padre.

Extracción de marginales. Una vez completadas ambas pasadas, la probabilidad marginal de cualquier consulta q asociada al nodo n_q se obtiene en $O(1)$:

$$\Pr(q \mid e) = \frac{w_q \cdot \mathrm{pd}(n_q)}{F(e)} \tag{18}$$

El Algoritmo 3 resume las dos pasadas y la extracción.

Algoritmo 3 Diferenciación de circuitos d-DNNF (Darwiche, 2003)

Entrada: Circuito con n nodos en orden topológico; pesos w_i ajustados por evidencia.
Salida: val[i] y pd[i] para todo nodo i; marginal de cualquier consulta en $O(1)$.

```
    // Pasada ascendente
 1: for i ← 1 a n do
 2:     if i es hoja then val[i] ← w_i
 3:     else if i es AND then val[i] ← ∏_{j∈hijos(i)} val[j]
 4:     else (i es OR) val[i] ← ∑_{j∈hijos(i)} val[j]
 5:     end if
 6: end for
    // Pasada descendente
 7: pd[raíz] ← 1
 8: for i ← n a 1 do
 9:     for cada hijo j de i do
10:         if i es OR then pd[j] += pd[i]
11:         else (i es AND) pd[j] += pd[i] × ∏_{k∈hijos(i), k≠j} val[k]
12:         end if
13:     end for
14: end for
    // Extracción de consultas
15: for cada consulta q con nodo n_q y peso w_q do
16:     Pr(q | e) ← w_q · pd[n_q] / val[raíz]
17: end for
```

Ilustración del paso descendente. Para concretar la regla del nodo AND, considérese un nodo con dos hijos c_1 y c_2 con valores $\mathrm{val}(c_1) = 0.7$ y $\mathrm{val}(c_2) = 0.6$. La pasada ascendente produce $\mathrm{val}(\mathrm{AND}) = 0.42$. Si $\mathrm{pd}(\mathrm{AND}) = 1$, la pasada descendente asigna:

$$\mathrm{pd}(c_1) = 1 \times 0.6 = 0.6 \qquad \mathrm{pd}(c_2) = 1 \times 0.7 = 0.7$$

El mensaje que recibe c_1 es el valor de su hermano c_2: si c_1 aumentase en una unidad, el producto aumentaría en 0.6 unidades, que es exactamente la derivada parcial del producto respecto de c_1. La regla del nodo OR es más directa: ambos hijos reciben el gradiente del padre sin modificar, pues la derivada de una suma respecto de cualquier sumando es siempre 1.

4.6.4. Implementación y activación

La implementación separa la estructura del circuito de su evaluación en dos clases.

`DDNNFTopology` es una caché inmutable que almacena el tipo de cada nodo, su lista de hijos y su número total. Se calcula una sola vez por circuito compilado y se comparte entre todas las evaluaciones posteriores. Su construcción verifica que los nodos estén en orden topológico estricto: para todo nodo interno, todos sus hijos tienen índice menor. Esta propiedad está garantizada por los compiladores de ProbLog y permite que ambas pasadas se implementen como bucles simples sin necesidad de ordenación explícita.

`DarwicheDDNNFEvaluator` es una subclase de `problog.evaluator.Evaluator` que implementa la interfaz estándar de ProbLog con el evaluador de diferenciación. Su método `propagate()` aplica la evidencia, ejecuta la pasada ascendente y luego la descendente. Su método `evaluate_all_queries(` extrae todas las marginales en $O(Q)$ mediante la Ecuación 18, sin ningún recorrido adicional del circuito.

El evaluador se activa mediante el parámetro `darwiche=True` al instanciar el `MDP`. Su uso está restringido al *backend* d-DNNF; no es compatible con SDD. La Tabla 8 resume la comparación de costo entre los dos evaluadores.

Tabla 8: Comparación de costo asintótico entre los evaluadores de ProbLog.

Operación	**Estándar**	**Darwiche**
Una sola consulta	$O(\|C\|)$	$O(1)$ tras propagación
Q consultas simultáneas	$O(Q \times \|C\|)$	$O(\|C\|) + O(Q)$
Iteración de Valor completa	$O(S \times A \times Q \times \|C\|)$	$O(S \times A \times \|C\|)$

La ganancia es proporcional a Q. Para un modelo con 8 consultas por par estado-acción ($Q = 8$), el costo de cada evaluación se reduce en un factor 8 respecto al evaluador estándar. En dominios con espacios de estados grandes o muchas utilidades, donde Q puede superar las decenas, esta reducción tiene un impacto directo en el tiempo de convergencia de la Iteración de Valor.

4.7. Herramientas auxiliares

Al converger la Iteración de Valor, el solver produce la función de valor óptima $V^*(s)$, la política óptima $\pi^*(s)$ y, si fue solicitada, la tabla $Q^*(s, a)$. Estas estructuras residen en memoria como objetos internos del *framework*. Para que el usuario pueda inspeccionarlas y validar la corrección del modelo resuelto, la extensión incorpora dos módulos de soporte: `CSVExporter` y `Simulator`.

4.7.1. Exportador de resultados

`CSVExporter` serializa las estructuras del MDP y los resultados de la Iteración de Valor a archivos en formato CSV. El exportador recibe como entradas el objeto `MDP` preparado y el objeto resultado de la Iteración de Valor. Expone un método por estructura exportada. Esta organización permite al usuario solicitar únicamente las salidas que necesite en cada análisis. El método `export_all(vi)` actúa como orquestador e invoca todos los métodos en secuencia, omitiendo las exportaciones opcionales cuando la estructura correspondiente no está disponible. Cada archivo generado comienza con un encabezado de metadatos formado por líneas con prefijo `#` que registran el tipo de contenido y la marca de tiempo de generación. La Tabla 9 lista los métodos y las estructuras que producen.

La exportación de la matriz de transición requiere un paso de expansión que las demás exportaciones no necesitan. El *framework* almacena transiciones en representación factorizada. Para cada par (s, a), el motor devuelve distribuciones marginales independientes por factor.

Tabla 9: Métodos de exportación del `CSVExporter` y estructuras producidas.

Método	Estructura exportada	Disponibilidad
`export_transition_matrix()`	$P(s' \mid s, a)$ plana	Siempre
`export_reward_matrix()`	$R(s, a)$	Siempre
`export_value_function(vi)`	$V^*(s)$	Siempre
`export_policy(vi)`	$\pi^*(s)$	Siempre
`export_q_table(vi)`	$Q^*(s, a)$	Si fue computada
`export_convergence(vi)`	Historial $V_k(s)$	Si fue registrado
`export_all(vi)`	Todas las anteriores	Orquestador

La matriz $P(s' \mid s, a)$, en cambio, requiere la probabilidad conjunta sobre todos los estados sucesores. Para obtenerla, el método `_expand_transitions()` computa el producto cartesiano de las distribuciones factorizadas. Esta función aplica la misma recursión sobre factores descrita en la Sección 4.5: en cada nivel k se itera sobre las ramas del factor k, se calcula el índice global acumulado mediante `get_local_index` y el *stride* S_k, y se recurre al nivel $k + 1$ con la probabilidad conjunta actualizada. Solo se emiten entradas con probabilidad estrictamente mayor que cero. Esta restricción mantiene el volumen del archivo reducido en modelos con transiciones dispersas.

4.7.2. Simulador de trayectorias

La política óptima $\pi^*(s)$ es un resultado analítico obtenido por programación dinámica. Una forma de verificar su calidad de forma independiente es ejecutarla contra la dinámica estocástica del modelo. El retorno acumulado que se obtiene en la práctica permite contrastar el resultado analítico con el comportamiento empírico. `Simulator` realiza esta verificación mediante rollouts.

`Simulator` recibe como entradas el MDP preparado y la política $\pi^* : \mathcal{S} \to \mathcal{A}$ como diccionario. El método principal es `run(trials, horizon, start_state, gamma)`. Ejecuta `trials` trayectorias independientes, cada una partiendo de `start_state`. Las recompensas se descuentan con el factor γ en cada paso y la trayectoria avanza hasta `horizon` pasos como máximo. La Tabla 10 describe los parámetros.

Tabla 10: Parámetros del método `Simulator.run()`.

Parámetro	Tipo	Descripción
`trials`	`int`	Número de trayectorias independientes a ejecutar.
`horizon`	`int`	Longitud máxima de cada trayectoria en pasos.
`start_state`	`tuple`	Estado inicial desde el que parten todas las trayectorias.
`gamma`	`float`	Factor de descuento $\gamma \in (0,\ 1]$.

En cada paso de tiempo, el simulador ejecuta cuatro operaciones en secuencia:

1. Consulta $\pi^*(s)$ para determinar la acción a prescrita en el estado actual s.

2. Invoca `structured_transition(s, a)` para obtener las distribuciones factorizadas $[\mathcal{F}_0, \ldots, \mathcal{F}_{m-1}]$.

3. Para cada factor k, muestrea el valor del estado sucesor mediante selección categórica ponderada sobre las ramas de $\mathcal{F}_k$.

4. Reconstruye la representación del estado sucesor usando `get_local_index`: los factores booleanos producen un valor en $\{0, 1\}$; los factores multivaluados producen la codificación *one-hot* del valor muestreado.

El muestreo opera factor a factor de forma independiente. Esta independencia es válida bajo el supuesto de independencia condicional entre factores establecido en la Ecuación 15: el valor sucesor de cada variable de estado es estadísticamente independiente del de las demás, dado el par (s, a). El método devuelve la recompensa promedio sobre todos los ensayos, la lista de recompensas individuales y las trayectorias completas. Las trayectorias individuales permiten al usuario examinar episodios concretos y no solo el retorno promedio.

`CSVExporter` y `Simulator` operan sobre la misma representación factorizada que utiliza el solver. Esta consistencia garantiza que la expansión de transiciones del exportador y el muestreo del simulador sean coherentes con los cálculos de la Iteración de Valor. La sección siguiente establece que el conjunto de modificaciones introducido en este capítulo preserva el comportamiento del *framework* sobre los modelos booleanos existentes.

4.8. Retrocompatibilidad

La extensión fue diseñada para preservar el comportamiento del *framework* ante programas que utilizan exclusivamente fluentes booleanos, sin requerir modificaciones por parte del usuario. Esta garantía se sustenta en el comportamiento del `FluentClassifier` descrito en la Sección 4.3. Cuando el programa no contiene Disyunciones Anotadas, no hay nodos `choice` que registrar en el índice invertido. La clasificación implícita de todo fluente declarado con `state_fluent/1` resulta entonces en tipo booleano.

Bajo esta clasificación, el `FluentSchema` registra cada fluente como un factor de base 2. El cálculo de *strides* para m factores booleanos produce la serie $\{1, 2, 4, \ldots, 2^{m-1}\}$. Esta serie es la codificación binaria estándar de posiciones. La inyección de hechos *dummy*, descrita en la Sección 4.4.2, inserta por cada fluente un hecho con probabilidad 0.5. El espacio de estados resultante tiene tamaño $|\mathcal{S}| = 2^m$ y se enumera como el producto cartesiano de m factores binarios. El comportamiento observado en cada etapa del *pipeline* es idéntico al de la arquitectura original.

La extensión también soporta modelos mixtos, donde algunos fluentes son booleanos y otros son multivaluados. El `FluentClassifier` trata cada tipo de forma independiente. Los fluentes booleanos se registran como factores de base 2 y los multivaluados como factores de base N. El usuario puede incorporar fluentes multivaluados en un programa existente añadiendo

únicamente las nuevas declaraciones. Las reglas de transición booleanas no requieren modificación. Esta característica permite adoptar la sintaxis multivaluada de manera gradual, sin necesidad de reescribir las partes del programa que ya son correctas.

Con la arquitectura, los algoritmos y la implementación definidos, el sistema cuenta con la capacidad de modelar y resolver MDPs con fluentes multivaluados de manera nativa. La propiedad de retrocompatibilidad garantiza que esta extensión no invalida ningún modelo booleano existente. El siguiente capítulo somete la extensión a una evaluación empírica mediante casos de estudio que cuantifican su impacto tanto en la simplificación del modelado como en el comportamiento computacional del solver.

5. Experimentación y Resultados

El capítulo anterior presentó el diseño y la implementación de la extensión al *framework* MDP-ProbLog. El presente capítulo somete dicha extensión a una evaluación empírica mediante un conjunto de experimentos diseñados para responder tres preguntas de investigación.

1. **RQ1 — Correctitud funcional.** ¿Producen la codificación binaria y la codificación factorizada la misma función de valor óptima V^* y la misma política π^* para el mismo MDP?

2. **RQ2 — Simplificación del modelado.** ¿Reduce la codificación factorizada la complejidad del programa ProbLog que el modelador debe escribir respecto a la codificación binaria?

3. **RQ3 — Impacto computacional.** ¿Cómo afecta la elección de codificación a los tiempos de construcción del MDP y de resolución por Iteración de Valor?

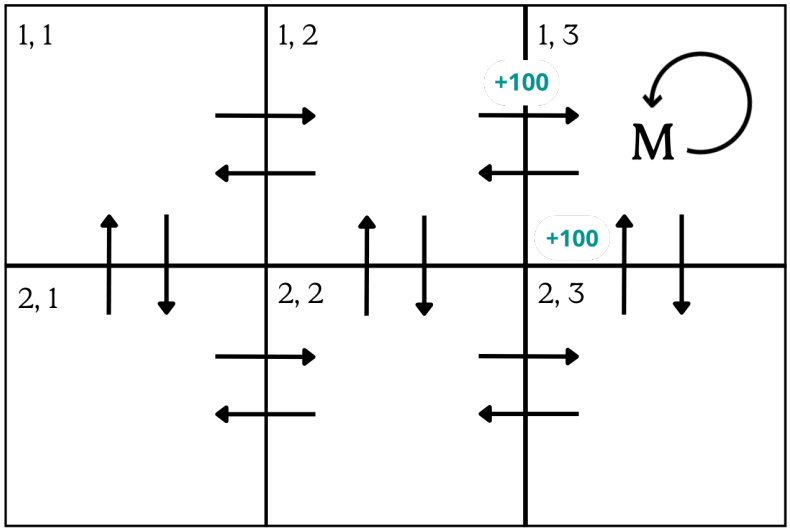
5.1. Diseño experimental

5.1.1. Dominios de evaluación

El dominio primario de evaluación es una familia de *grids* basada en el *grid world* de Mitchell [9]. En este dominio, un agente navega por una cuadrícula rectangular de R filas y C columnas. El agente dispone de cinco acciones: `left`, `right`, `up`, `down` y `stay`. Las transiciones son deterministas: cada acción produce el movimiento esperado si la celda destino existe dentro de los límites del *grid*. Si el agente intenta moverse contra un muro, permanece en su celda. Una celda terminal absorbente en la esquina superior derecha otorga una recompensa de $+100$ al ser alcanzada. El dominio es escalable porque la estructura de las reglas de transición es independiente del tamaño del *grid*.

Se seleccionaron ocho tamaños de *grid* para el análisis de escalamiento: 2×3, 3×3, 4×4, 6×6, 8×8, 12×12, 16×16 y 32×32. El tamaño 2×3 es el *grid* original de Mitchell y sirve como punto de verificación contra la solución publicada. Los tamaños intermedios permiten observar las tendencias de escalamiento. El tamaño 32×32 representa el límite superior evaluado, con 1 024 celdas.

El dominio secundario es la instancia canónica del *grid* de Russell y Norvig [23]. Este dominio consiste en un *grid* 3×4 con un obstáculo en la celda $(2, 2)$, un estado meta en $(3, 4)$ con recompensa $+1$, un estado trampa en $(2, 4)$ con recompensa -1, y un costo por paso de -0.04 para todos los estados no terminales. Las transiciones son estocásticas: la acción elegida produce el movimiento deseado con probabilidad 0.80, y con probabilidad 0.10 el agente se desplaza hacia cada uno de los dos lados perpendiculares. Si el movimiento resultante choca con un muro, el agente permanece en su celda. El dominio Russell se evalúa en una única instancia canónica de 11 estados para verificar la corrección de la extensión contra los valores publicados en la Figura 17.3 de Russell y Norvig [23].



Acciones: izquierda, derecha, arriba, abajo y quedarse.

Figura 7: Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [9].

5.1.2. Variables independientes

El experimento manipula dos variables independientes. La primera es la **codificación del espacio de estados**. La codificación binaria (BSF) representa la posición del agente mediante $\lceil \log_2(R \times C + 1) \rceil$ fluentes booleanos independientes. Cada fluente toma el valor 0 o 1, y la combinación de todos ellos identifica el estado. La codificación factorizada (AD) representa la posición mediante dos fluentes multivaluados, uno para la fila y uno para la columna, declarados como Disyunciones Anotadas. El espacio de estados resultante tiene exactamente $R \times C$ elementos sin estados espurios. La codificación binaria se aplica únicamente al dominio Mitchell; la codificación factorizada se aplica a ambos dominios.

La segunda variable es el ***backend* de compilación**. Se evalúan tres *backends* disponibles en ProbLog. El *backend ddnnf* compila el programa aterrizado en un circuito d-DNNF y evalúa cada consulta $P(f \mid s, a)$ mediante un recorrido independiente del circuito, con costo $O(|\mathcal{S}| \times |\mathcal{A}|)$ por iteración de Bellman. El *backend* darwiche compila el mismo circuito d-DNNF pero calcula todas las marginales de forma simultánea mediante el esquema de diferenciación de Darwiche [20], reduciendo el costo por iteración a $O(|C|)$ independientemente del número de fluentes consultados. El *backend sdd* compila el programa en un Diagrama de Decisión Sentencial (SDD) [19] y evalúa las consultas mediante el evaluador estándar de ProbLog.

5.1.3. Protocolo de medición

El experimento comprende 50 configuraciones: 48 del dominio Mitchell (8 tamaños $\times$ 2 codificaciones $\times$ 3 *backends*) y 2 del dominio Russell (codificación factorizada con *backends ddnnf*

y darwiche). Cada configuración se ejecuta $N = 11$ veces de forma independiente. La primera ejecución se descarta como calentamiento para estabilizar el entorno de ejecución. Las 10 ejecuciones restantes constituyen las mediciones efectivas. En cada ejecución se ejecuta el pipeline completo desde cero: *parsing*, clasificación de fluentes, *grounding*, compilación del circuito e Iteración de Valor.

El límite de *timeout* es de 400 s por ejecución individual. Si una ejecución supera ese límite, se registra con estado `TIMEOUT` y los datos parciales disponibles. Los parámetros del solver se fijan en $\gamma = 0.9$ para el dominio Mitchell y $\gamma = 1.0$ para el dominio Russell, con $\varepsilon = 0.01$ y $\varepsilon_{\text{thr}} = 10^{-6}$. Estos parámetros se mantienen constantes en todas las configuraciones de cada dominio. Los tiempos se miden con `time.perf_counter()` dentro de cada ejecución.

5.2. Entorno de evaluación

Los experimentos se ejecutaron sobre un equipo con arquitectura x86_64 y sistema operativo Linux 6.17.0-20-generic. El intérprete utilizado fue Python 3.12.3. Las mediciones se realizaron en ejecuciones seriales sin paralelismo; en cada ejecución el proceso disponía de todos los recursos del equipo sin competencia de otros procesos.

El experimento comprende 50 configuraciones y 550 ejecuciones totales (50×11). De las 50 configuraciones, 43 completaron al menos 10 ejecuciones exitosas dentro del límite de *timeout*. Las 7 configuraciones restantes registraron estado `TIMEOUT` en todas sus ejecuciones medidas. Las configuraciones que alcanzaron el *timeout* corresponden a la codificación binaria en *grids* de 12×12 o mayores (con *backend ddnnf*) y a la codificación factorizada en el *grid* 32×32 con *backend ddnnf*. El *timeout* es un resultado experimental válido que documenta los límites de escalabilidad de cada configuración.

5.3. RQ1 — Corrección funcional

El primer objetivo de la evaluación es demostrar que la extensión multivaluada produce resultados correctos. Para ello se comparan las soluciones obtenidas por ambas codificaciones entre sí y, en el dominio Russell, contra la solución publicada de referencia.

5.3.1. Verificación en el dominio Mitchell

La Tabla 11 presenta los resultados de la verificación cruzada entre codificaciones para el dominio Mitchell. Para cada tamaño de *grid* donde ambas codificaciones completaron, se comparan los valores numéricos de $V^*(s)$ para todos los estados del espacio de la codificación factorizada, y se verifica si las políticas $\pi^*(s)$ son idénticas. La tolerancia de comparación numérica es 10^{-6}.

Las diferencias máximas entre ambas codificaciones son exactamente 0.0 en todos los tamaños evaluados, y las políticas son idénticas en cada caso. El número de iteraciones de Bellman hasta convergencia es igual para ambas codificaciones en cada tamaño. Esta igualdad se sigue de que ambas codificaciones representan el mismo MDP: la estructura de transiciones y recompensas es idéntica, y el algoritmo de Iteración de Valor aplicado a estructuras isomorfas

Tabla 11: Verificación cruzada de corrección funcional en el dominio Mitchell. La columna máx $|\Delta V^*|$ reporta la diferencia máxima absoluta entre las funciones de valor de ambas codificaciones. La columna Iter. reporta el número de iteraciones de Bellman hasta convergencia, que es idéntico para ambas codificaciones en cada tamaño.

Grid	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fac}}$	máx $\lvert\Delta V^*\rvert$	π^* **coincide**	**Iter.**
2×3	8	6	0.0	Sí	3
3×3	16	9	0.0	Sí	3
4×4	32	16	0.0	Sí	4
6×6	64	36	0.0	Sí	6
8×8	128	64	0.0	Sí	8
12×12	256	144	0.0	Sí	12
16×16	512	256	0.0	Sí	16

El *grid* 32×32 no aparece porque la codificación binaria alcanzó el *timeout* en las tres combinaciones de *backend* evaluadas; la verificación cruzada requiere que ambas codificaciones completen.

converge en el mismo número de pasos con los mismos parámetros γ y ε [3]. La verificación del dominio Russell se presenta en la Sección 5.6.

5.4. RQ2 — Simplificación del modelado

La segunda pregunta de investigación evalúa si la codificación factorizada reduce la complejidad del programa ProbLog que el modelador debe escribir. Las métricas consideradas son el número de fluentes declarados, las cardinalidades de sus dominios, el número de reglas en el programa fuente, el tamaño del espacio de estados $|\mathcal{S}|$ y la cantidad de estados espurios generados.

La Tabla 12 presenta estas métricas estáticas para las dos codificaciones en los ocho tamaños de *grid* evaluados.

Los datos revelan tres regularidades. La primera es la constancia de la codificación factorizada. Independientemente del tamaño del *grid*, la codificación factorizada genera siempre un programa con 2 fluentes y 21 reglas. Las cardinalidades de los dos fluentes coinciden exactamente con las dimensiones del *grid* ($R \times C$), lo que hace explícita la correspondencia entre la representación lógica y la estructura del dominio. Esta constancia se debe a que las reglas de transición están parametrizadas por variables lógicas y aritmética, de modo que el mismo programa describe la dinámica para cualquier dimensión del *grid*.

La segunda regularidad es el crecimiento superlineal de la codificación binaria. El número de reglas crece de 42 (en 2×3) a 5 376 (en 16×16), un factor de $128\times$ para un incremento de $43\times$ en el número de celdas. Este crecimiento proviene de que cada transición requiere reglas individuales por estado, y el número de estados crece exponencialmente con el número de bits.

La tercera regularidad concierne al *grid* 4×4. La codificación binaria utiliza 5 bits para

Tabla 12: Métricas estáticas de los programas ProbLog generados por cada codificación. Para la codificación binaria, la columna Cardinalidades indica el número de fluentes booleanos. Para la factorizada, indica las cardinalidades de los dos fluentes multivaluados.

Grid	**Cod.**	**Fluentes**	**Cardinalidades**	**Reglas**	$\lvert\mathcal{S}\rvert$	**Esp.**
2×3	Bin.	3	3 bits	42	8	2
	Fac.	2	2×3	21	6	0
3×3	Bin.	4	4 bits	77	16	7
	Fac.	2	3×3	21	9	0
4×4	Bin.	5	5 bits	178	32	16
	Fac.	2	4×4	21	16	0
6×6	Bin.	6	6 bits	472	64	28
	Fac.	2	6×6	21	36	0
8×8	Bin.	7	7 bits	1 025	128	64
	Fac.	2	8×8	21	64	0
12×12	Bin.	8	8 bits	2 595	256	112
	Fac.	2	12×12	21	144	0
16×16	Bin.	9	9 bits	5 376	512	256
	Fac.	2	16×16	21	256	0
32×32	Bin.†	11	11 bits	26 623	2 048†	1 024†
	Fac.	2	32×32	21	1 024	0

† Las métricas $\lvert\mathcal{S}\rvert$ y espurios de la fila 32×32 binario son valores teóricos derivados del número de bits ($2^{11} = 2\,048$ estados, $2\,048 - 1\,024 = 1\,024$ espurios). Las tres configuraciones de *backend* alcanzaron el *timeout* de 400 s sin producir resultados de tiempo.

representar 16 celdas, generando un espacio de $2^5 = 32$ estados con 16 estados espurios. El mínimo teórico sería 4 bits ($2^4 = 16$ estados exactos). La discrepancia se debe a que la implementación reserva el estado 0 (todos los fluentes a cero) para uso interno, desplazando todos los índices de celda en una unidad. Esta restricción produce siempre al menos un estado espurio adicional sobre el mínimo teórico cuando el número de celdas es una potencia exacta de 2.

5.5. RQ3 — Impacto computacional

La tercera pregunta de investigación evalúa cómo influye la elección de codificación sobre el rendimiento del solver. Cada configuración combina una de las dos codificaciones con uno de los tres *backends*. El tiempo de ejecución se descompone en dos fases. La fase de construcción (t_{build}) abarca el *parsing* del programa fuente, la clasificación de fluentes, el *grounding* y la compilación del circuito. La fase de Iteración de Valor (t_{VI}) ejecuta las actualizaciones de Bellman hasta convergencia. La métrica primaria reportada es el tiempo total $t_{\text{total}} = t_{\text{build}} + t_{\text{VI}}$.

5.5.1. Tiempos de ejecución

La Tabla 13 presenta t_{total} para cada combinación de codificación y *backend*. Las configuraciones que excedieron el límite de 400 s se marcan como T.O. Para las configuraciones que completaron, se reporta la media y la desviación estándar sobre 10 ejecuciones independientes.

Tabla 13: Tiempo total de ejecución t_{total} (segundos) por codificación y *backend*. Se reporta media ± desviación estándar sobre 10 ejecuciones. T.O. indica que la configuración superó el límite de 400 s.

Grid	**Cod.**	**ddnnf** (s)	**darwiche** (s)	**sdd** (s)
2×3	Bin.	0.218 ± 0.117	0.109 ± 0.005	0.056 ± 0.003
	Fac.	0.237 ± 0.014	0.103 ± 0.008	0.059 ± 0.018
3×3	Bin.	0.805 ± 0.037	0.379 ± 0.016	0.100 ± 0.001
	Fac.	0.504 ± 0.015	0.175 ± 0.029	0.067 ± 0.005
4×4	Bin.	4.37 ± 0.05	1.52 ± 0.03	0.315 ± 0.036
	Fac.	1.14 ± 0.02	0.283 ± 0.017	0.100 ± 0.003
6×6	Bin.	27.8 ± 2.6	6.98 ± 0.13	1.11 ± 0.04
	Fac.	4.22 ± 0.14	0.701 ± 0.031	0.244 ± 0.028
8×8	Bin.	131.8 ± 6.4	32.9 ± 1.2	4.20 ± 0.57
	Fac.	10.8 ± 0.6	1.40 ± 0.04	0.507 ± 0.033
12×12	Bin.	T.O.	161 ± 3.4	20.5 ± 1.3
	Fac.	68.9 ± 3.1	5.36 ± 0.12	1.82 ± 0.05
16×16	Bin.	T.O.	T.O.	94.2 ± 10.2
	Fac.	245 ± 2.3	14.8 ± 0.6	5.11 ± 0.46
32×32	Bin.	T.O.	T.O.	T.O.
	Fac.	T.O.	189 ± 10	58.9 ± 3.8

Los tiempos de la codificación binaria crecen más rápido que los de la factorizada a medida que aumenta el tamaño del *grid*. Para el *backend* darwiche, la codificación binaria pasa de 0.109 s en 2×3 a 161 s en 12×12, un factor de 1 477×. En ese mismo rango, la codificación factorizada crece de 0.103 s a 5.36 s, un factor de 52×.

La fase de Iteración de Valor domina t_{total} en la mayoría de las configuraciones. Para la codificación factorizada con *backend* darwiche, el tiempo de construcción representa el 15.7 % del total en 2×3 y el 2.0 % en 16×16. La proporción decrece con el tamaño del *grid* porque el número de evaluaciones de $Q(s, a)$ durante la Iteración de Valor crece con $|\mathcal{S}| \times |\mathcal{A}|$, mientras que el costo de construcción del circuito crece más lentamente.

5.5.2. Cociente de tiempos entre codificaciones

La Tabla 14 presenta el cociente $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ para los pares de configuraciones donde ambas codificaciones completaron. Un valor mayor que 1.0 indica que la codificación factorizada finalizó en menos tiempo.

Tabla 14: Cociente $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ por *backend*. El símbolo ∞ indica que la codificación binaria superó el *timeout* mientras la factorizada completó. Un valor < 1 indica que la binaria fue más rápida.

Grid	**ddnnf**	**darwiche**	**sdd**
2×3	0.92×	1.06×	0.95×
3×3	1.60×	2.17×	1.49×
4×4	3.82×	5.36×	3.15×
6×6	6.59×	9.96×	4.56×
8×8	12.2×	23.6×	8.28×
12×12	∞	30.0×	11.2×
16×16	∞	∞	18.4×

Para el *grid* 2×3, la codificación binaria es ligeramente más rápida o comparable en *ddnnf* y *sdd* (0.92× y 0.95× respectivamente). Para *darwiche*, la codificación factorizada ya es 1.06× más rápida en ese tamaño. A partir del *grid* 3×3, la codificación factorizada es más rápida en las tres combinaciones de *backend*, y el cociente crece de forma monotónica con el tamaño.

El cociente máximo observado en pares donde ambas codificaciones completaron es 30.0×, para el *backend* darwiche en 12×12. La inversión de rendimiento entre 2×3 y 3×3 responde al costo fijo de las Disyunciones Anotadas. Para *grids* pequeños, el *overhead* de construir y compilar los nodos *choice* supera la ventaja de un espacio de estados reducido. Para *grids* de 3×3 en adelante, la reducción del espacio de estados y del número de cláusulas compensa ese costo fijo.

5.5.3. Frontera de escalabilidad

La Tabla 15 resume el tamaño máximo de *grid* para el que cada combinación de codificación y *backend* completó dentro del límite de *timeout*.

Tabla 15: Tamaño máximo de *grid* que completó dentro del límite de 400 s, por codificación y *backend*. Entre paréntesis se indica el número de estados del espacio de estados correspondiente.

Codificación	**ddnnf**	**darwiche**	**sdd**
Binaria	8×8 (128)	12×12 (256)	16×16 (512)
Factorizada	16×16 (256)	32×32 (1 024)	32×32 (1 024)

La codificación factorizada extiende la frontera de escalabilidad en todos los *backends*. Para *ddnnf*, la frontera pasa de 8×8 (binaria) a 16×16 (factorizada), duplicando el número máximo de estados. Para darwiche y *sdd*, la codificación factorizada permite resolver el *grid* 32×32 con 1 024 estados, cuatro veces más que el máximo alcanzado por la codificación binaria con esos *backends*.

La codificación binaria para el *grid* 32×32 genera 11 fluentes booleanos y un espacio de $2^{11} = 2\,048$ estados, con 1 024 estados espurios. Las 26 623 reglas del programa resultan en un programa aterrizado de tamaño suficiente como para que los tres *backends* superen el límite de 400 s sin completar la Iteración de Valor.

5.5.4. Comparación de *backends*

Los tres *backends* difieren en velocidad absoluta y en la tasa de crecimiento con el tamaño del *grid*. El análisis se realiza sobre la codificación factorizada, que ofrece el mayor número de configuraciones completadas.

Para la codificación factorizada, el *backend sdd* produce los tiempos más bajos en todos los tamaños evaluados. El *backend* darwiche es entre 1.75× y 3.2× más lento que *sdd*, con la diferencia creciendo lentamente con el tamaño. El *backend ddnnf* muestra la tasa de crecimiento más alta. En el *grid* 4×4, *ddnnf* es 4.0× más lento que darwiche; en el *grid* 16×16, esa diferencia es 16.5×.

La divergencia entre *ddnnf* y los otros dos *backends* se explica por la estructura computacional de cada uno. El *backend ddnnf* requiere compilar un circuito d-DNNF separado para cada par (s, a) consultado durante la Iteración de Valor. Los *backends* darwiche y *sdd* compilan el circuito una sola vez y derivan todas las probabilidades de transición mediante diferenciación o consulta dirigida sobre esa misma estructura. Esta diferencia reduce el número de compilaciones de $O(|\mathcal{S}| \times |\mathcal{A}|)$ a $O(1)$ por iteración de Bellman.

5.5.5. Desglose de la fase de construcción

La fase de construcción t_{build} se divide en cuatro subfases: *parsing* del programa fuente (t_{parse}), clasificación de fluentes (t_{classify}), *grounding* del programa (t_{ground}) y compilación del circuito (t_{compile}). La Tabla 16 presenta estas subfases para la codificación factorizada con el *backend* darwiche.

Tabla 16: Desglose de t_{build} en subfases (segundos) para la codificación factorizada con *backend* darwiche. Se reportan medias sobre 10 ejecuciones.

Grid	t_{parse}	t_{classify}	t_{ground}	t_{compile}	t_{build}
2×3	0.0039	0.0006	0.0237	0.0150	0.044
3×3	0.0038	0.0006	0.0245	0.0274	0.057
4×4	0.0037	0.0007	0.0372	0.0206	0.063
6×6	0.0039	0.0008	0.0516	0.0237	0.081
8×8	0.0041	0.0011	0.0825	0.0280	0.117
12×12	0.0041	0.0014	0.1434	0.0456	0.196
16×16	0.0044	0.0017	0.2165	0.0720	0.296
32×32	0.0051	0.0031	0.6028	0.2058	0.818

El *grounding* es la subfase dominante de t_{build}. Su participación relativa crece del 53.8 % en 2×3 al 73.7 % en 32×32. El tiempo de *grounding* pasa de 0.024 s en 2×3 a 0.603 s en 32×32, un factor de 25× para un incremento de 171× en $|\mathcal{S}|$. Esta sublinealidad se debe a que el programa fuente contiene exactamente 21 reglas en todos los tamaños. El *grounding* solo debe instanciar esas reglas para los valores concretos de fila y columna del *grid*.

El *parsing* y la clasificación permanecen prácticamente constantes en todos los tamaños evaluados. El tiempo de *parsing* varía entre 0.0037 s y 0.0051 s, y el de clasificación entre 0.0006 s y 0.0031 s. La compilación crece de 0.015 s a 0.206 s, representando entre el 25.2 % y el 34.1 % de t_{build}.

5.6. Dominio Russell

El dominio de Mitchell utiliza transiciones deterministas, lo que aísla el efecto de la representación sobre el rendimiento pero no prueba la extensión en condiciones estocásticas. El dominio de Russell complementa esta evaluación: el *grid* canónico 3×4 de Russell y Norvig [23] incorpora transiciones probabilísticas, un estado trampa y un costo por paso, ofreciendo un escenario de complejidad cualitativa distinta. La evaluación se realiza exclusivamente con la codificación factorizada, que expresa la posición del agente como un único fluente multivaluado `coor(X,Y)` con 11 valores posibles. El programa fuente para este dominio contiene 50 reglas y 66 líneas. Los *backends* evaluados son *ddnnf* y darwiche; el *backend sdd* produjo un error de compilación en este dominio y no se incluye en el análisis cuantitativo.

5.6.1. Verificación de la función de valor óptima

La función de valor óptima $V^*(s)$ se verificó comparando los valores producidos por MDP-ProbLog contra los valores de referencia publicados en la Figura 17.3 de Russell y Norvig [23]. El *grid* 3×4 utiliza los parámetros canónicos del libro: $\gamma = 1.0$, costo por paso $R = -0.04$, transiciones con probabilidad 0.80 en la dirección deseada y 0.10 hacia cada lado perpendicular. La Tabla 17 presenta los 9 estados no terminales del dominio con sus valores de referencia y las diferencias absolutas respecto al valor obtenido por el solver.

Tabla 17: Verificación de $V^*(s)$ para el dominio Russell 3×4. Los valores de referencia corresponden a la Figura 17.3 de Russell y Norvig [23]. La columna $|\Delta V^*|$ reporta la diferencia absoluta entre el valor obtenido por MDP-ProbLog y el valor de referencia.

Fila	**Columna**	V^* **(referencia)**	$\lvert\Delta V^*\rvert$
3	1	0.812	< 0.001
3	2	0.868	< 0.001
3	3	0.918	< 0.001
2	1	0.762	< 0.001
2	3	0.660	< 0.001
1	1	0.705	< 0.001
1	2	0.655	< 0.001
1	3	0.611	< 0.001
1	4	0.388	< 0.001
3	4	meta $(+1)$	0.000
2	4	trampa (-1)	0.000

Las coordenadas (X, Y) siguen la convención del modelo: fila $X = 1$ en la base del *grid*, columna $Y = 1$ a la izquierda. El estado $(2, 2)$ es un obstáculo y no pertenece al espacio de estados. Los estados terminales tienen $V^* = 0$ por convención de estado absorbente: la recompensa ± 1 se recibe al entrar al estado, no al permanecer en él.

Los nueve estados no terminales presentan diferencias menores a 0.001 respecto a los valores de referencia. Los estados terminales tienen $V^* = 0$ en el solver, consistente con la semántica de absorción. La política óptima π^* obtenida coincide con la Figura 17.2 de Russell y Norvig [23]: los estados de las filas superiores dirigen al agente hacia el estado meta, mientras que los estados de la fila inferior evitan el estado trampa desplazándose hacia la izquierda.

El número de iteraciones de Bellman hasta convergencia fue 43. Este valor es considerablemente mayor que el observado en los *grids* de Mitchell de tamaño comparable, donde el *grid* 4×4 convergió en 4 iteraciones. La diferencia se explica por dos factores. El primero es el factor de descuento: Russell usa $\gamma = 1.0$, lo que elimina el amortiguamiento geométrico de los valores entre iteraciones. El segundo es el criterio de convergencia: con $\gamma = 1$ y el criterio max_residual $\leq 2\varepsilon(1-\gamma)/\gamma$, el umbral se reduce a 0, exigiendo convergencia exacta al punto fijo [3].

5.6.2. Tiempos de ejecución

La Tabla 18 presenta los tiempos de construcción, Iteración de Valor y total para los dos *backends* evaluados en el dominio Russell.

El *backend* darwiche completa la ejecución en 0.280 s, frente a 1.094 s del *backend ddnnf*, un cociente de 3.91×. La diferencia se origina casi exclusivamente en la fase de Iteración de Valor: t_{VI} es 4.78× menor con darwiche. Los tiempos de construcción son equivalentes para ambos *backends* (0.061 s frente a 0.064 s), lo que confirma que los dos comparten el mismo compilador d-DNNF y difieren únicamente en el evaluador.

Tabla 18: Tiempos de ejecución para el dominio Russell 3×4 con codificación factorizada. Se reporta media $\pm$ desviación estándar sobre 10 ejecuciones.

Backend	t_{build} (s)	t_{VI} (s)	t_{total} (s)
ddnnf	0.061 ± 0.003	1.032 ± 0.009	1.094 ± 0.009
darwiche	0.064 ± 0.006	0.216 ± 0.001	0.280 ± 0.006
Cociente ddnnf/darwiche	$0.95\times$	$4.78\times$	$3.91\times$

El cociente de rendimiento entre *backends* para Russell ($3.91\times$) es inferior al observado en *grids* de Mitchell de tamaño comparable. El *grid* 4×4 de Mitchell con codificación factorizada mostró un cociente *ddnnf*/darwiche de $4.04\times$, y el *grid* 8×8 de $7.75\times$. Esta diferencia se debe a que el número de evaluaciones por iteración en Russell es $|\mathcal{S}| \times |\mathcal{A}| = 11 \times 5 = 55$, menor que el de los *grids* de Mitchell con espacio de estados equivalente. El circuito compilado del dominio Russell es más complejo por la naturaleza estocástica de las transiciones, pero el número de consultas por iteración es más reducido.

5.7. Discusión

5.7.1. RQ1 — Correctitud funcional

Ambas codificaciones producen la misma función de valor óptima V^* y la misma política π^* para todos los tamaños de *grid* evaluados en el dominio Mitchell. Las diferencias máximas entre codificaciones son exactamente 0.0 en todos los casos (Tabla 11). RQ1 se responde afirmativamente: la extensión multivaluada preserva la semántica del MDP subyacente sin introducir desviaciones en la solución.

Un aspecto no evidente de este resultado es la igualdad en el número de iteraciones de Bellman hasta convergencia. Para cada tamaño de *grid*, ambas codificaciones convergen en el mismo número de iteraciones, desde 3 en 2×3 hasta 16 en 16×16. Esta igualdad se sigue de que ambas codificaciones representan el mismo MDP. La estructura del proceso de decisión, es decir, el conjunto de estados, las funciones de transición y las recompensas, es idéntica en las dos formulaciones porque la representación cambia la etiqueta de los estados pero no la geometría del proceso. Con los mismos parámetros γ y ε, el algoritmo de Iteración de Valor aplicado a dos formulaciones isomorfas del mismo MDP converge en el mismo número de pasos [3].

La verificación en el dominio Russell extiende este resultado a transiciones estocásticas. Los nueve estados no terminales del *grid* canónico 3×4 mostraron diferencias menores a 0.001 respecto a la solución de referencia de AIMA (Tabla 17). La política obtenida coincide con la Figura 17.2 de Russell y Norvig [23]. Este resultado confirma que la extensión maneja correctamente las Disyunciones Anotadas bajo distribuciones de probabilidad no triviales, incluyendo la redistribución de masa hacia muros adyacentes.

5.7.2. RQ2 — Simplificación del modelado

La codificación factorizada produce un programa de tamaño constante en todos los tamaños de *grid* evaluados. El programa tiene 2 fluentes y 21 reglas independientemente del tamaño, desde 2×3 hasta 32×32 (Tabla 12). Esta constancia es consecuencia directa de la naturaleza paramétrica de las reglas de transición. Al expresar la dinámica en términos de coordenadas y aritmética sobre variables lógicas, una sola regla captura el comportamiento de una acción a través de todas las celdas del *grid*. El modelador no necesita modificar el programa al escalar el dominio. RQ2 se responde afirmativamente.

La codificación binaria, en contraste, requiere enumeración explícita. El número de reglas crece de 42 en 2×3 a 5 376 en 16×16, un factor de $128\times$ para un incremento de $43\times$ en el número de celdas. Este crecimiento superlineal se origina en que cada transición debe describirse en términos de qué bits se activan, y el número de bits necesarios crece logarítmicamente con el número de celdas. Al escalar el dominio, el modelador debe reescribir el programa completo para cada nuevo tamaño.

Un caso particular ilustra una restricción de la codificación binaria. El *grid* 4×4 tiene 16 celdas, para las que el mínimo teórico es 4 bits. La implementación usa 5 bits y genera 32 estados con 16 espurios porque el estado 0 no puede emplearse. Esta restricción implica que la codificación binaria siempre produce al menos un estado espurio adicional sobre el mínimo teórico cuando el número de celdas es exactamente una potencia de 2. La codificación factorizada no tiene esta limitación porque el dominio de cada fluente se declara explícitamente y no requiere un valor de reserva.

5.7.3. RQ3 — Impacto computacional

El efecto de la codificación sobre el rendimiento sigue un patrón consistente en los tres *backends*. Para el *grid* 2×3, la codificación binaria es comparable o ligeramente más rápida (cocientes de $0.92\times$, $1.06\times$ y $0.95\times$ para *ddnnf*, darwiche y *sdd* respectivamente). A partir del *grid* 3×3, la codificación factorizada es más rápida en todas las combinaciones. El punto de inversión refleja el costo fijo de las Disyunciones Anotadas: para *grids* pequeños, el *overhead* de compilar los nodos *choice* supera la ventaja de un espacio de estados reducido. A partir de ese umbral, la reducción del espacio de estados y del número de cláusulas compensa ese costo fijo y el cociente crece de forma monotónica con el tamaño. RQ3 se responde afirmativamente para *grids* de 3×3 en adelante.

El cociente de rendimiento es mayor para el *backend* darwiche que para *sdd* o *ddnnf* a igual tamaño de *grid* (Tabla 14). Esta amplificación se debe a la interacción entre la codificación factorizada y el evaluador de diferenciación. El evaluador darwiche calcula todas las marginales del circuito en dos recorridos por par (s, a), independientemente del número de fluentes consultados. Con la codificación factorizada, el espacio de estados es menor que con la binaria, por lo que el número de pares (s, a) evaluados por iteración es también menor. Los dos efectos se combinan: menos pares a evaluar, y cada par evaluado con igual eficiencia. El *backend sdd* logra tiempos absolutos menores, pero el efecto de la codificación sobre el cociente es más pronunciado con darwiche.

La consecuencia más directa de la ventaja computacional es la extensión de la frontera de escalabilidad. Con el *backend* darwiche, la codificación binaria alcanza como máximo el *grid* 12×12 (256 estados) mientras que la factorizada resuelve el *grid* 32×32 (1 024 estados) dentro del mismo límite de tiempo (Tabla 15). Esta diferencia no es consecuencia de ninguna optimización del solver de Iteración de Valor sino exclusivamente del tamaño del espacio de estados y del programa que cada codificación genera.

5.7.4. Análisis de cuellos de botella

El tiempo total de ejecución está dominado por la Iteración de Valor en la mayoría de las configuraciones. Para la codificación factorizada con *backend* darwiche, t_{VI} representa el 84.3 % del total en 2×3 y el 98.0 % en 16×16. Esta proporción crece con el tamaño del *grid* porque el número de evaluaciones de $Q(s, a)$ por iteración escala con $|\mathcal{S}| \times |\mathcal{A}|$, mientras que el costo de construcción del circuito crece más lentamente.

Dentro de la fase de construcción, el *grounding* es la subfase dominante. Su participación crece del 53.8 % de t_{build} en 2×3 al 73.7 % en 32×32 (Tabla 16). El tiempo de *grounding* pasa de 0.024 s a 0.603 s para un incremento de 171× en $|\mathcal{S}|$, lo que indica que la fase es sublineal en el número de estados. El *parsing* y la clasificación son prácticamente constantes porque operan sobre el programa fuente, que tiene 21 reglas en todos los tamaños.

La estructura de los cuellos de botella identifica las oportunidades prioritarias de optimización. Reducir el costo de cada evaluación $Q(s, a)$ durante la Iteración de Valor tendría el mayor impacto sobre t_{total}, y el *backend* darwiche ya aborda este punto mediante diferenciación simultánea. Reducir el costo del *grounding* tendría el mayor impacto sobre t_{build}, que en las configuraciones factorizadas representa entre el 2 % y el 15.7 % del tiempo total. El *parsing* y la clasificación no representan cuellos de botella en ningún tamaño evaluado.

5.7.5. Limitaciones de la evaluación

La evaluación cubre un conjunto representativo de escenarios pero no es exhaustiva en la variedad de dominios. Todas las instancias del dominio Mitchell son *grids* rectangulares sin obstáculos, con transiciones deterministas y una única fuente de recompensa. Este diseño aísla efectivamente el efecto de la codificación, pero no permite generalizar los resultados a dominios con múltiples fluentes de estado, transiciones estocásticas entre estados no adyacentes, o funciones de recompensa dependientes del contexto.

El dominio Russell introduce estocasticidad pero se evalúa en una sola instancia canónica de 11 estados. No es posible derivar tendencias de escalamiento para ese dominio a partir de los datos recopilados. Generar instancias Russell de mayor tamaño requeriría redistribuir las probabilidades de transición celda por celda según los obstáculos adyacentes, lo que introduce fuentes de variación adicionales respecto a la solución de referencia publicada.

La incompatibilidad del *backend sdd* con el dominio Russell no se diagnosticó en el marco de este experimento. Los circuitos SDD producidos para dominios con Disyunciones Anotadas estocásticas pueden tener propiedades estructurales distintas a los circuitos d-DNNF, y la

causa específica del fallo no fue identificada. Dado que *sdd* es el *backend* más rápido para el dominio Mitchell, el comportamiento de ese *backend* en dominios estocásticos queda como cuestión abierta.

Finalmente, los resultados del dominio Mitchell corresponden a una única instancia por tamaño de *grid*. No se evalúan variaciones en la posición del estado meta, la presencia de obstáculos, el número de acciones disponibles, ni la función de recompensa. Estos factores podrían afectar el número de iteraciones hasta convergencia, el tamaño del circuito compilado, o la relación de rendimiento entre codificaciones para *grids* del mismo tamaño. El capítulo siguiente integra los resultados de la evaluación con el diseño de la extensión para formular las conclusiones del trabajo y plantear las líneas de investigación futura.

Índice de figuras

1. Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3. 3
2. Grid de navegación 2x3 estocástico. Se incorporan transiciones probabilísticas. 14
3. Política óptima: (a) una flecha indica la acción óptima $\pi^*(r, c)$; (b) muestra el valor $V^*(r, c)$ calculado por Iteración de Valor. 23
4. Diagrama de bloques de la arquitectura original del framework MDP-ProbLog. 27
5. pipeline de clasificación. Se explica la ejecución del método `classify()` . . . 34
6. pipeline de preparación. Se explica la ejecución del método `__prepare()` . . . 36
7. Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [9]. 49

Índice de cuadros

1. Codificación binaria de las 6 celdas de un grid de 2×3. 3
2. Escalamiento de estados espurios en la codificación binaria por tamaño de grid. 4
3. Codificación binaria forzada para una variable ternaria en el dominio *Factory*. 5
4. Comparativa de los enfoques del estado del arte. 12
5. Mundos posibles inducidos por la semántica de distribución sobre el Código 1. 18
6. Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible. 20
7. Codificación *mixed-radix* para el dominio color $\times$ velocidad. 31
8. Comparación de costo asintótico entre los evaluadores de ProbLog. 44
9. Métodos de exportación del `CSVExporter` y estructuras producidas. 45
10. Parámetros del método `Simulator.run()`. 45
11. Verificación cruzada de corrección funcional en el dominio Mitchell. La columna máx $|\Delta V^*|$ reporta la diferencia máxima absoluta entre las funciones de valor de ambas codificaciones. La columna Iter. reporta el número de iteraciones de Bellman hasta convergencia, que es idéntico para ambas codificaciones en cada tamaño. 51
12. Métricas estáticas de los programas ProbLog generados por cada codificación. Para la codificación binaria, la columna Cardinalidades indica el número de fluentes booleanos. Para la factorizada, indica las cardinalidades de los dos fluentes multivaluados. 52
13. Tiempo total de ejecución t_{total} (segundos) por codificación y *backend*. Se reporta media $\pm$ desviación estándar sobre 10 ejecuciones. T.O. indica que la configuración superó el límite de 400 s. 53
14. Cociente $t^{\text{bin}}_{\text{total}}/t^{\text{fac}}_{\text{total}}$ por *backend*. El símbolo ∞ indica que la codificación binaria superó el *timeout* mientras la factorizada completó. Un valor < 1 indica que la binaria fue más rápida. 54
15. Tamaño máximo de *grid* que completó dentro del límite de 400 s, por codificación y *backend*. Entre paréntesis se indica el número de estados del espacio de estados correspondiente. 55
16. Desglose de t_{build} en subfases (segundos) para la codificación factorizada con *backend* darwiche. Se reportan medias sobre 10 ejecuciones. 56
17. Verificación de $V^*(s)$ para el dominio Russell 3×4. Los valores de referencia corresponden a la Figura 17.3 de Russell y Norvig [23]. La columna $|\Delta V^*|$ reporta la diferencia absoluta entre el valor obtenido por MDP-ProbLog y el valor de referencia. 57
18. Tiempos de ejecución para el dominio Russell 3×4 con codificación factorizada. Se reporta media $\pm$ desviación estándar sobre 10 ejecuciones. 58

Índice de algoritmos

1. Inferencia de tipo por intersección de orígenes estocásticos 35
2. Valor esperado del estado sucesor sobre el esquema factorizado 40
3. Diferenciación de circuitos d-DNNF (Darwiche, 2003) 43

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. «ProbLog: A probabilistic prolog and its application in link discovery». En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[3] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[4] Thiago P Bueno et al. «Markov decision processes specified by probabilistic logic programming: Representation and solution». En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[5] Craig Boutilier, Thomas Dean y Steve Hanks. «Decision-theoretic planning: Structural assumptions and computational leverage». En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[6] Jesse Hoey et al. «SPUDD: Stochastic Planning using Decision Diagrams». En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[7] Davide Nitti, Vaishak Belle y Luc De Raedt. «Planning in discrete and continuous Markov decision processes by probabilistic programming». En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[8] Héctor Avilés et al. «Autonomous Behavior Selection For Self-driving Cars Using Probabilistic Logic Factored Markov Decision Processes». En: *Applied Artificial Intelligence* 38.1 (2024), pág. 2304942.

[9] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997.

[10] Richard S Sutton y Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[11] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. «Logic programs with annotated disjunctions». En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[12] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. «Stochastic dynamic programming with factored representations». En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[13] Scott Sanner. «Relational Dynamic Influence Diagram Language (RDDL): Language Description». Unpublished Manuscript, Australian National University. 2010.

[14] Luc De Raedt y Angelika Kimmig. «Probabilistic (logic) programming concepts». En: *Machine Learning* 100.1 (2015), págs. 5-47.

[15] Guy Van den Broeck et al. «DTProbLog: A Decision-Theoretic Probabilistic Prolog». En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[16] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. «CP-logic: A language of causal probabilistic events and its relation to logic programming». En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[17] David Poole. «The Independent Choice Logic for Modelling Multiple Agents under Uncertainty». En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[18] Adnan Darwiche y Pierre Marquis. «A knowledge compilation map». En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[19] Adnan Darwiche. «SDD: A New Canonical Representation of Propositional Knowledge Bases». En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[20] Adnan Darwiche. «A differential approach to inference in Bayesian networks». En: *Journal of the ACM* 50.3 (2003), págs. 280-305.

[21] Anton Dries et al. «ProbLog2: Probabilistic Logic Programming». En: *Machine Learning and Knowledge Discovery in Databases*. Ed. por Albert Bifet et al. Cham: Springer International Publishing, 2015, págs. 312-315.

[22] Daan Fierens et al. «Inference and learning in probabilistic logic programs using weighted Boolean formulas». En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[23] Stuart Jonathan Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. English. Third. England: Pearson Education, 2016. ISBN: 9781292153964.

[24] Taisuke Sato. «A statistical learning method for logic programs with distribution semantics». En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.